

Summary2

Riassumimi in maniera precisa questo pdf, Rendi il discorso fluido e molto esteso.
nota che deve essere in inglese ed esportato su obsidian come md.
Non ricopiare link o riferimenti, il testo deve essere pulito.
Prendi tutti gli argomenti ed esponili in maniera estensiva e facile da capire

A. Parallel Algorithms and Parallel Programming

1. Automatic vs. Manual Parallelization

Automatic Parallelization

The idea of **automatic parallelization** is to write a standard sequential algorithm using sequential high-level code and rely entirely on automatic tools to handle the conversion to a parallel implementation.

However, this approach is considered (at the moment) **not feasible**. Automatic tools often fail because they cannot extract all available parallelism from a specification designed for sequential execution.

Example of Failure:

A compiler cannot safely parallelize loops in a function if it cannot determine whether pointer arguments (e.g., and, or, b, c) overlap in memory. The designer might never use the function in a way that causes overlap, but the compiler cannot know this.

Parallelization by Hand

This approach requires the programmer to be actively involved.

- The programmer must first design a **parallel algorithm**.
- This is then implemented using **parallel high-level code**.
- The programmer provides "hints" to the tools, leaving only the final code compilation to them.

Critical aspects of this manual process include:

1. Identifying which **type of parallelism** to use.
2. Deciding how to **design the parallel algorithm** (e.g., modifying a sequential one or starting from scratch).
3. Determining how to **provide this parallelism information** to the tools.

2. Types of Parallelism

1) Bit-Level Parallelism

This form of parallelism involves *manipulating different data represented by the bits composing a single word*. A single instruction can operate on all these bits at the same time. It is highly relevant in hardware implementation and can also be significant in software, such as using bit strings to represent sets.

2) Instruction-Level Parallelism (ILP)

ILP refers to executing different instructions at the same time *on the same core*. This is supported by architectural features like multiple execution units, pipelines, and SIMD units. This type of parallelism is often easily extracted by compilers.

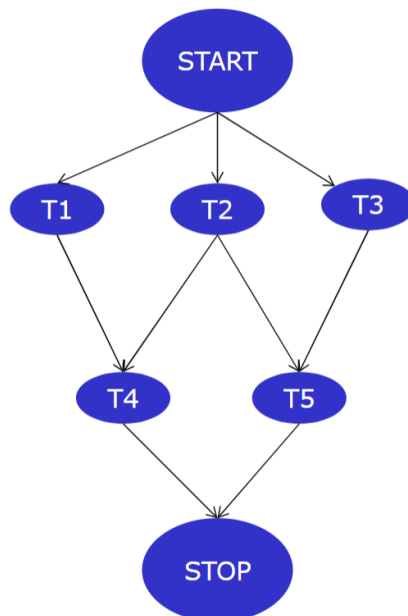
3) Task-Level Parallelism

A **task** is defined as a logically discrete section of computational work. Task-level parallelism involves a *parallel program running multiple tasks on multiple processors*. This is supported by mechanisms like shared memory and cache coherence but is *usually difficult to extract automatically*.

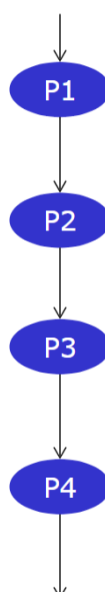
Tipo di Parallelismo	Descrizione	Caratteristiche Chiave / Supporto	Facilità di Estrazione
Bit-Level Parallelism	Manipolazione simultanea di diversi dati rappresentati dai bit di una singola parola.	Una singola istruzione opera su tutti i bit contemporaneamente. Rilevante in hardware e software (es. stringhe di bit per set).	Non specificato, ma intrinseco all'hardware.
Instruction-Level Parallelism (ILP)	Esecuzione di diverse istruzioni contemporaneamente <i>sullo stesso core</i> .	Supportato da unità di esecuzione multiple, pipeline e unità SIMD.	Spesso facilmente estraibile dai compilatori.
Task-Level Parallelism	Un programma parallelo esegue più compiti (sezioni discrete di lavoro computazionale) su più processori.	Supportato da meccanismi come memoria condivisa e coerenza della cache.	Di solito difficile da estrarre automaticamente.

Task-level models include:

1. **Parallel Task Graph:** Tasks are represented by vertices, and the edges represent precedencies or data communications.



2. **Pipeline:** Similar to a processor pipeline, this is well-suited for streaming data like audio or video encoding. Edges represent data passing between stages.



Communication between tasks is handled by two main models:

1. **Shared Memory:** All processors and tasks share a single global memory with the same address space. Modifications by one processor are visible to all others.
2. **Message Passing:** Each task has its own private memory. Tasks must communicate by *explicitly* sending and receiving messages.

Example of All Levels:

If the **restrict** keyword is used in the `and_or` function, the compiler knows the memory locations do not overlap. This allows for extracting parallelism at all levels:

- Task: The two loops (one for `and`, one for `or`) can be run in parallel.
- Instruction: Each iteration of a single loop can be run in parallel.
- Bit: The bitwise operations (`&` and `|`) can be computed in parallel.

Spiegazione

1. Il **restrict** keyword:

1. **restrict** è una parola chiave in C (e C++) che può essere usata con i puntatori.
2. Quando un programmatore dichiara un puntatore con **restrict** (es. `int *restrict ptr`), sta fornendo una garanzia al compilatore: per tutta la durata del puntatore, **ptr** sarà l'unico modo per accedere alla regione di memoria a cui punta.
3. Questo significa che nessun altro puntatore (o alias) accederà o modificherà quella stessa regione di memoria in modo indipendente.

2. "the compiler knows the memory locations do not overlap" (il compilatore sa che le posizioni di memoria non si sovrappongono):

1. Senza **restrict**, il compilatore deve essere cauto. Se una funzione riceve due puntatori, ad esempio `void func(int *a, int *b)`, il compilatore non può sapere se **a** e **b** puntano a regioni di memoria completamente diverse o se, in qualche caso, potrebbero puntare alla stessa posizione (questo è chiamato "aliasing").
2. Se **a** e **b** potessero sovrapporsi, una modifica tramite `*a` potrebbe influenzare il valore letto o scritto tramite `*b`, e viceversa. Questo crea una dipendenza di dati che impedisce al compilatore di riordinare o eseguire in parallelo le operazioni.
3. Usando **restrict**, il programmatore *garantisce* che i puntatori passati alla funzione (come quelli per le operazioni `and` e `or` nell'esempio) si riferiscono a blocchi di memoria distinti e non sovrapposti. Questa garanzia è cruciale.

3. "This allows for extracting parallelism at all levels" (Questo permette di estrarre parallelismo a tutti i livelli):

1. Una volta che il compilatore sa con certezza che le operazioni su diverse regioni di memoria non interferiranno tra loro, può applicare ottimizzazioni e parallelizzazioni molto più aggressive.
2. **Task-Level Parallelism:** Se la funzione `and_or` ha due cicli separati, uno per l'operazione `and` e uno per l'operazione `or`, e questi cicli operano su dati garantiti come non sovrapposti (grazie a **restrict** sui puntatori di input/output), il compilatore può decidere di eseguire questi due cicli in parallelo come compiti distinti su core diversi. Non c'è rischio che un ciclo modifichi i dati che l'altro sta usando in modo inaspettato.
3. **Instruction-Level Parallelism (ILP):** All'interno di un singolo ciclo (ad esempio, il ciclo per l'operazione `and`), se ogni iterazione opera su elementi di un array che sono garantiti come non sovrapposti (ancora grazie a **restrict**), il compilatore può parallelizzare le singole istruzioni. Può usare unità di esecuzione multiple, pipeline o istruzioni SIMD (Single Instruction, Multiple Data) per elaborare più elementi dell'array contemporaneamente, perché sa che le modifiche a un elemento non influenzeranno gli altri elementi elaborati in parallelo.
4. **Bit-Level Parallelism:** Le operazioni bitwise (`&` e `|`) sono intrinsecamente parallele a livello di bit. Il **restrict** keyword, assicurando che i dati su cui si opera possano essere caricati e memorizzati in modo sicuro e indipendente, permette all'hardware di sfruttare appieno questa parallelizzazione a livello di bit senza preoccupazioni di dipendenze o conflitti di memoria.

In sintesi, il **restrict** keyword è uno strumento potente che il programmatore usa per comunicare intenzioni precise al compilatore riguardo all'uso della memoria. Questa informazione aggiuntiva rimuove ambiguità e "paure" che il compilatore altrimenti avrebbe, sbloccando la possibilità di applicare ottimizzazioni di parallelizzazione a tutti i livelli, dal singolo bit fino a interi compiti complessi.

3. Parallel Programming

Design and Architecture Challenges

Simply designing a good parallel algorithm is not enough. **Not all extracted parallelism is exploitable on a real architecture.**

- The programmer must consider the parallelism **available** on the target architecture.
- Using an unsuitable type of parallelism **can introduce overhead**.
- The parallelism must be described to the compilation tools to make it exploitable.

This contrasts sharply with sequential programming. In a sequential world, compilers can effectively manage the gap between different processors (e.g., an old Intel 80386 vs. a new Intel i9) and optimize the **same** source code for both.

In parallel programming, **real architectures differ so much that compilers cannot fill this gap**. Optimized applications cannot be generated from generic parallel code. This has led to two main approaches:

1. **New Programming Languages:** These were developed for the new parallel paradigm but did not see wide success due to immature compilers and the high learning curve.
2. **Extensions to Existing Languages:** This is the more successful method. These extensions are easily adopted by designers and integrated into existing compilers. Their drawback is that they can often only describe **some** types of parallelism.

This specialization for particular applications and architectures is why so many parallel programming languages and APIs exist today.

TECHNOLOGY	TYPE	YEAR
Verilog/VHDL	Languages	1984/1987
MPI	Library	1994
PThread	Library	1995
OpenMP	C/Fortran Extensions	1997
CUDA	C Extensions	2007
OpenCL	C/C++ Extensions + API	2008
Apache Spark	API	2014

4. Comparison of Parallel Programming Technologies (**SKIP**)

Target Architecture and Memory

- **Verilog/VHDL:** Targets ASICs and FPGAs, using mainly distributed memory.
- **MPI:** Targets multi-CPU, using distributed memory.
- **PThread:** Targets multi-CPU, using mainly shared memory.
- **OpenMP:** Targets multi-core CPU, using mainly shared memory.
- **CUDA:** Targets CPU + NVIDIA GPU; memory is distributed between the CPU and GPU, but is shared **on** the GPU.
- **OpenCL:** Targets heterogeneous architectures (CPU, GPU, etc.), supporting both distributed and shared memory.
- **Apache Spark:** Targets multi-CPU, using (distributed) shared memory.

Parallelism and Communication Model

- **Explicit Parallelism, Explicit Communication:** Verilog/VHDL.
- **Explicit Parallelism, Implicit Communication:** PThread, OpenMP.
- **Implicit Parallelism, Explicit Communication:** MPI.
- **Implicit Parallelism, Implicit Communication:** Apache Spark.
- **Mixed (CUDA & OpenCL):** Parallelism and communication **between** the CPU and a kernel are explicit. Parallelism and communication **within** a kernel (among threads) are implicit.

A key takeaway is that even when a technology supports target-independent code, **performance optimization usually requires knowledge of the target architecture**.

5. Pros and Cons of Key Technologies (**SKIP**)

Verilog/VHDL

- **Pros:** Complete control over computation and memory, no computational overhead, access to massive computational power.
- **Cons:** Requires specific hardware (ASIC/FPGA), difficult to learn (a completely different paradigm), dependent on the target architecture.

MPI (Message Passing Interface)

- **Pros:** Can be used on different architectures, provides scalable solutions, synchronization and communication are explicitly managed.
- **Cons:** Communication can introduce significant overhead, programming paradigm is more difficult than shared-memory models.

PThread (POSIX Threads)

- **Pros:** Can be used on different architectures, offers explicit parallelism and full control.
- **Cons:** Task management overhead can be significant, solutions are not easily scalable, it is a low-level API.

OpenMP

- **Pros:** Easy to learn, provides scalable solutions, parallel applications can also be executed sequentially.
- **Cons:** Mainly focused on shared memory, homogeneous systems, requires minimal interaction between tasks.

CUDA

- **Pros:** Provides access to the high computational power of GPUs, writing a basic kernel is quite easy, many optimized libraries are available.
- **Cons:** Targets *only* NVIDIA GPUs, it can be difficult to extract massive parallelism from an application, it is difficult to *optimize* a kernel.

OpenCL

- **Pros:** It is a target-independent standard that hides architecture details, provides a single infrastructure for very heterogeneous systems (CPU+GPU+FPGA).
- **Cons:** The paradigm can be difficult due to its heterogeneity, and the hiding of architecture details makes it difficult to achieve the best performance.

Apache Spark

- **Pros:** Provides APIs for different languages, does not require explicit parallelization or communication, often preinstalled on cloud provider VMs.
- **Cons:** Suitable *only* for big data applications, does not (yet) fully support GPUs.

6. Mixing Parallelism Technologies (**SKIP**)

It is common to combine technologies to exploit different architectural features.

- **OpenMP + CUDA:** Used to exploit both multi-core CPUs and GPUs. OpenMP is used to parallelize the CPU code, and CUDA is used to parallelize the GPU code.
- **MPI + OpenMP:** Can be used in two ways:
 1. MPI for coarser-grained parallelism (across multiple CPUs) and OpenMP for finer-grained parallelism (across cores on a single CPU).
 2. MPI to implement communications while OpenMP is used to parallelize the computation.
- **OpenCL + Verilog/VHDL:** Hardware kernels (on an FPGA) can be used as accelerators. OpenCL is used to describe the parallelism *among* different processing elements, while Verilog/VHDL is used to describe the *hardware kernel itself*.

7. Domain-Specific Languages (DSLs) (**SKIP**)

An example given is **Halide**, a DSL for image processing and deep learning.

- Its key concept is the **separation of the algorithm from its execution schedule**.
- The *algorithm* defines *what* to compute.

- The *schedule* defines *how* to compute it (e.g., loop nesting, parallelization, vector instructions).
- This allows the programmer to experiment with many different schedules to find the most efficient one for a given architecture.

B. Design Guidelines for Parallel Programming

Introduction: The Challenge of Parallel Algorithms

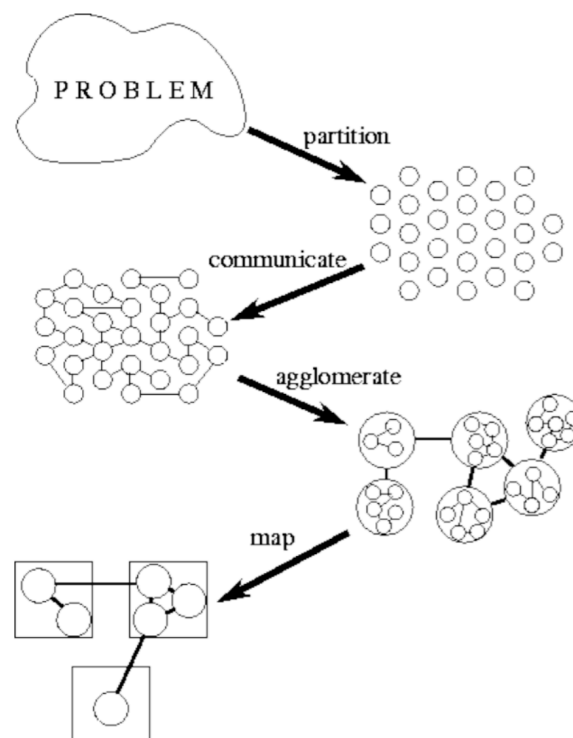
Designing effective parallel algorithms is an inherently complex task that *lacks a single, universal formula for success*. It is a creative process that requires careful consideration of the problem's nature. While there are no strict algorithms for creating parallel algorithms, a *set of structured design guidelines can help navigate the complexities*. The key is to adopt a methodical approach that begins with machine-independent concepts like concurrency and gradually incorporates target-specific architectural details.

A robust methodology for this process is known as **PCAM**, which stands for *Partitioning, Communication, Agglomeration, and Mapping*.

These four stages guide the developer from the initial problem analysis to an efficient parallel implementation.

The PCAM Design Methodology

The PCAM methodology provides a structured framework for designing parallel programs. It breaks the process into four distinct, though often overlapping, stages.



1. Partitioning: Exposing Parallelism

The primary goal of the partitioning stage is to *break down a problem into a large number of small, independent tasks*. This process, known as **decomposition**, is intended to expose the maximum possible opportunities for parallel execution. By creating a fine-grained decomposition, we create a flexible foundation for the later stages of the design. *A good partition divides both the computational work and the data it operates on into small, manageable pieces, ideally without replicating data or computation.*

There are *two primary approaches to partitioning*:

- **Domain (Data) Decomposition:** This strategy focuses on the *data associated with a problem*. The dataset is divided into smaller, typically equal-sized portions, and each parallel task is assigned to work on one portion. This

is a very common technique, especially in scientific computing where large data structures like arrays or matrices are manipulated.

- **Functional Decomposition:** This approach focuses on the computation itself rather than the data. The problem is *decomposed according to the different types of work that must be performed*. For example, in a signal processing pipeline, one task might be responsible for filtering the signal, another for encoding it, and a third for compressing it. Each function becomes a task that can potentially be executed in parallel. An example of this is ecosystem modeling, where the population of each species (plants, herbivores, carnivores) is calculated by a separate task, with each task depending on the state of its neighbors.

In practice, a combination of both domain and functional decomposition can often yield the best results.

2. Communication: Coordinating Tasks

Once the problem has been partitioned into tasks, those tasks will inevitably need to communicate and coordinate to solve the larger problem. This stage involves defining the necessary information flow between tasks. Communication is often the most critical aspect of parallel program design and a primary source of overhead that can nullify the benefits of parallelization.

Key aspects of communication include:

- **Communication Types:**
 - **Local vs. Global:** Does a task need to communicate with only a small set of neighbors or with many tasks across the system?
 - **Structured vs. Unstructured:** Do communications follow a regular, predictable pattern (e.g., a grid or tree) or are they arbitrary and irregular?
 - **Static vs. Dynamic:** Can the communication partners be determined at compile time, or do they change based on runtime conditions?
- **Communication Patterns:**
 - **Point-to-Point:** A simple exchange of information between two tasks, one sender and one receiver.
 - **Collective:** Communication involving more than two tasks within a group. Common collective operations include broadcasting (one task sending data to all others), scattering (one task sending a portion of its data to each of the others), gathering (each task sending data to one designated task), and reduction (gathering data and combining it with an operation like sum or max).
 - Broadcast:** One task sends the same piece of data to all other tasks in the group.
 - Scatter:** One task distributes different portions of its data to the other tasks.
 - Gather:** The inverse of scatter, where one task collects data from all others in the group.
 - Reduction:** A form of gather where the collected data is combined using an operation (e.g., sum, max, min).

The characteristics of the target architecture heavily influence how communication is implemented, whether through explicit message passing in distributed-memory systems or implicit communication through shared memory.

The characteristics of communication are heavily dependent on the **target architecture**:

- **Visibility:** Communication can be **explicit**, as in *message passing* systems, where the programmer must write specific code to send and receive messages. Alternatively, it can be **implicit**, as in *shared-memory* systems, where tasks communicate by reading from and writing to common memory locations.
- **Synchronization:** It can be **synchronous**, requiring a "handshake" between tasks (a sending operation blocks until the receiver is ready), or **asynchronous**, where tasks can transfer data independently without blocking and waiting for each other.

The Cost of Communication: Overhead

When moving to the implementation phase on a specific architecture, communication becomes the most critical aspect. The **communication overhead**—the time spent communicating instead of computing—can be so significant that it can **completely nullify the benefits** gained from parallelization. This overhead is primarily caused by two factors:

- **Transmission:** The actual time it takes to move data across the network or system bus.
- **Synchronization:** The time tasks spend waiting for each other to coordinate.

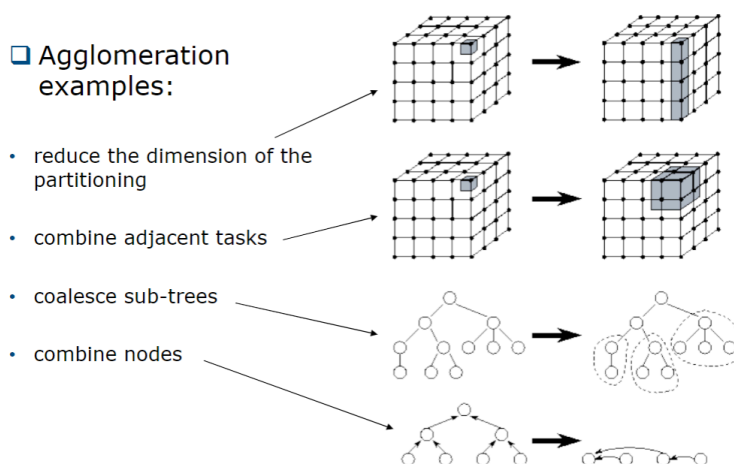
For this reason, a key design goal is to minimize communication and organize it efficiently. One powerful technique is to **overlap communication with computation**, allowing a task to perform other useful calculations while it waits for data to arrive, thus hiding the communication latency.

3. Agglomeration: From Abstraction to Reality

The partitioning stage aims to create a very large number of fine-grained tasks to expose maximum parallelism. However, in a real-world implementation, *creating and managing too many tasks can introduce significant overhead from task creation and communication*. The agglomeration stage addresses this by moving from the abstract parallel algorithm toward a more practical implementation.

In this phase, the fine-grained tasks from the partitioning step are grouped, or *agglomerated*, into larger, more coarse-grained tasks. The primary goals of agglomeration are:

- **Reduce Communication Costs:** By combining tasks that communicate frequently into a single, larger task, communication that was previously inter-process now becomes intra-process, which is significantly faster. This is related to *improving the "surface-to-volume" ratio of tasks, where the amount of computation (volume) is large relative to the amount of communication required (surface)*.
- **Increase Granularity:** Larger tasks generally have a higher ratio of computation to communication, which improves efficiency.
- **Maintain Scalability:** While reducing the number of tasks, it's crucial to ensure that there are still enough tasks to *keep all available processors busy* and that the number of tasks can scale with the problem size.



Agglomeration involves revisiting the partitioning and communication patterns to strike a balance between parallelism and overhead for a specific class of parallel architectures.

4. Mapping: Assigning Tasks to Processors

The final stage is mapping, which involves **assigning each agglomerated task to a specific processor for execution**. The ultimate goal of mapping is to **minimize total execution time**. This often involves balancing two conflicting objectives:

1. **Maximize Concurrency:** Place concurrent tasks on *different* processors so they can execute in parallel.
2. **Increase Locality:** Place tasks that communicate frequently on the *same* processor, or on processors that are close to each other in the network, to **reduce communication latency and cost**.

Finding the optimal mapping is an **NP-complete problem** (Non-deterministic Polynomial time), meaning that finding a perfect solution is computationally infeasible for all but the simplest cases. Therefore, heuristics and algorithms are used to find good-enough solutions. Key strategies include:

- **Load Balancing:** Distribute the tasks as evenly as possible among processors to ensure that no single processor becomes a bottleneck while others sit idle. This can be done statically at the beginning of the program or dynamically as the program runs.
- **Task Scheduling:** In designs based on functional decomposition, a task scheduling algorithm may be used to assign tasks to available processors to keep them busy.

This stage is highly dependent on the target architecture, as shared-memory systems have different mapping concerns than distributed-memory systems.

Spiegazione

- **NP** sta per "Non-deterministic Polynomial time". Significa che, se ti viene data una potenziale soluzione a un problema NP, puoi verificarne la correttezza in un tempo ragionevole (tempo polinomiale).
- **Completo** significa che il problema è tra i più difficili all'interno della classe NP. Se si trovasse un modo efficiente (in tempo polinomiale) per risolvere un singolo problema NP-completo, allora si potrebbe risolvere **ogni** problema nella classe NP in modo efficiente.

"Finding the optimal mapping is an NP-complete problem, meaning that finding a perfect solution is computationally infeasible for all but the simplest cases. Therefore, heuristics and algorithms are used to find good-enough solutions."

Questo significa che:

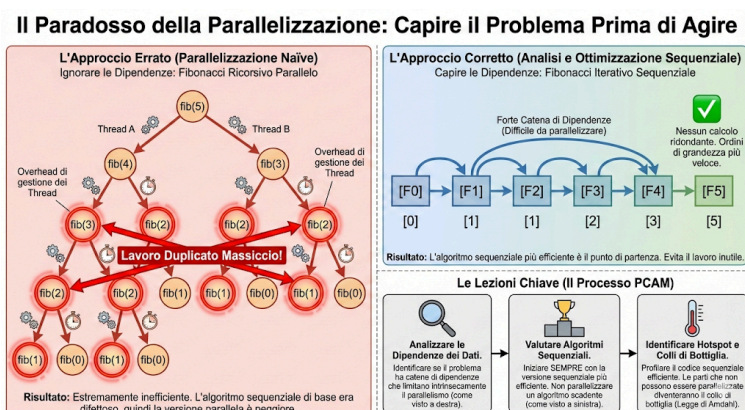
1. **Trovare la soluzione perfetta è quasi impossibile:** Per la maggior parte dei problemi NP-completi, non esiste un algoritmo conosciuto che possa trovare la soluzione migliore in un tempo ragionevole (cioè, un tempo che cresce in modo polinomiale con la dimensione del problema). Il tempo necessario per trovare la soluzione ottimale cresce esponenzialmente, rendendolo impraticabile per problemi di dimensioni anche moderate.
2. **Si usano soluzioni "abbastanza buone":** Poiché trovare la soluzione ottimale è troppo costoso in termini di tempo di calcolo, per i problemi NP-completi si ricorre spesso a:
 - **Euristiche:** Metodi che trovano soluzioni buone, ma non garantite come ottimali, in un tempo ragionevole.
 - **Algoritmi di approssimazione:** Algoritmi che garantiscono di trovare una soluzione entro una certa percentuale dalla soluzione ottimale, sempre in un tempo ragionevole.

In sintesi, un problema NP-completo è un problema per cui è facile verificare una soluzione, ma estremamente difficile (e spesso impossibile in pratica) trovare la soluzione ottimale in modo efficiente.

From Theory to Practice: Understanding the Problem First

Before even beginning the PCAM process, it is paramount to **deeply understand the problem** and its underlying data dependencies. A flawed sequential algorithm will only become a flawed and inefficient parallel one.

The **Fibonacci sequence** provides a classic example. A naive recursive implementation can be "parallelized" by computing `fib(n-1)` and `fib(n-2)` in parallel. However, this approach is extremely inefficient because it performs a massive number of redundant calculations. A simple iterative, sequential algorithm is orders of magnitude faster because it avoids these repeated operations and has no parallel overhead.



This illustrates a critical lesson:

1. **Analyze Data Dependencies:** First, identify if the problem has dependencies that inherently limit parallelism. The Fibonacci calculation has a strong chain of dependencies.

2. **Evaluate Sequential Algorithms:** Always start with the most efficient sequential algorithm. Most successful parallel algorithms are derived from efficient sequential versions. Attempting to design a parallel algorithm from scratch without this foundation is a very difficult task.
 3. **Identify Hotspots and Bottlenecks:** Profile the sequential code to find the "*hotspots*"—the parts where most of the computation occurs. These are the primary candidates for parallelization. If a hotspot cannot be parallelized, it will become a "*bottleneck*" that limits the overall speedup, as described by Amdahl's Law.
-

C. POSIX Threads and OpenMP: A Guide to Shared-Memory Parallel Programming

Introduction to Parallel Programming Concepts

Parallel programming is a paradigm that moves beyond sequential execution to leverage the power of modern multi-core and multi-processor hardware. To understand the tools used for this, it's essential to differentiate between a few key concepts:

- **Parallel Architectures:** These describe the physical hardware resources, such as CPUs, memories, and the interconnections between them.
 - The two primary memory architectures are **Shared Memory**, where all CPUs have access to a common pool of memory, and **Distributed Memory**, where each CPU has its own private memory, and communication occurs over a network.
- **Parallel Programming Models:** These are abstractions that sit above the hardware, providing a conceptual framework for the programmer.
 - Models like the **Threads Model** or the **Message Passing Model** define how parallel tasks are created, how they communicate, and how they synchronize. These models do not necessarily have to match the underlying hardware architecture.
- **Implementations:** These are the concrete tools—libraries or language extensions—that realize a programming model. **POSIX Threads (Pthreads)** and **OpenMP** are two popular implementations of the shared-memory parallel programming model using threads.

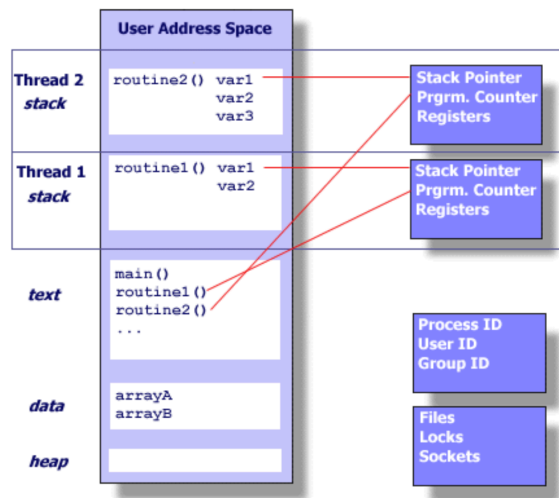
The Threads Model of Parallelism

The threads model is one of the most widely used paradigms for parallel programming on shared memory systems. It is based on the concept of a **thread**, which is an independent stream of instructions that can be scheduled and executed by the operating system.

Processes vs. Threads

To understand a thread, it is useful to contrast it with a **process**.

- A **process** is a self-contained execution environment created by the operating system. It includes a complete set of resources, such as a unique address space (for code, data, and heap), file handles, and process IDs. Creating a process is a relatively "heavyweight" operation.
- A **thread** exists *within* a process. Multiple threads can exist in a single process, and they share most of the process's resources, including the address space (code and global variables) and file descriptors. *Each thread, however, has its own private resources, most notably its own stack* (for local variables) and its own instruction pointer and registers.



Because *threads* share so many resources, they *are considered "lightweight" compared to processes*. They can be created more quickly and communication between them is implicit and fast—one thread can write to a variable, and another can immediately read the updated value.

The Challenge: Data Races and the Need for Synchronization

This implicit communication via shared memory is both a strength and a major challenge. When multiple threads concurrently access the same memory location, and at least one of those accesses is a write, *a data race or race condition can occur*. The final outcome of the program can become non-deterministic, depending on the precise order in which the threads are scheduled by the operating system.

For example, if two threads both execute the statement `s = s + value`, the result might be incorrect because one thread could read the value of `s` before the other thread has finished writing its updated value back to memory.

To prevent data races and ensure program correctness, the programmer must use **explicit synchronization**. This involves inserting mechanisms into the code that control access to shared resources, ensuring that only one thread can modify a critical piece of data at any given time.

POSIX Threads (Pthreads): The Low-Level Library Approach

Pthreads is a standard (IEEE POSIX 1003.1c) that defines an API for creating and managing threads. It is a C-based library that gives the programmer explicit, fine-grained control over all aspects of a thread's life and interactions.

The Pthreads API

The Pthreads API is divided into two main categories:

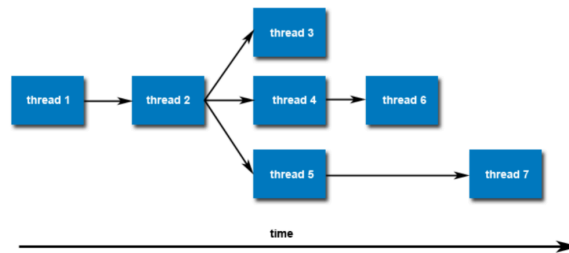
1. **Thread Management:** Routines for controlling the lifecycle of threads.
2. **Synchronization:** Routines for managing concurrent access to shared resources.

1. Thread Lifecycle Management

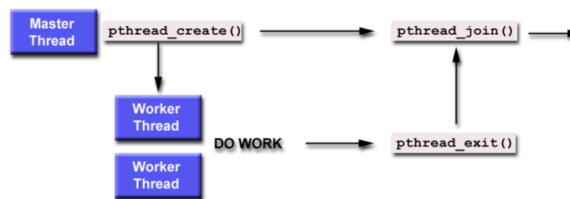
- **Creation:** Threads are created using the `pthread_create()` function. This function takes a pointer to the function the new thread will execute (the `start_routine`) and any arguments it needs. Once created, threads are peers; there is no inherent hierarchy.

```
int pthread_create(pthread_t * thread, const pthread_attr_t * attr, void * (*
start_routine) (void *), void *arg)
```

C++



- **Termination:** A thread can terminate in several ways: by returning from its start routine, by calling `pthread_exit()`, by being canceled by another thread `pthread_cancel()`, or if the entire process terminates.
- **Joining:** A common pattern is *for a main thread to create several worker threads and then wait for them to complete their work*. The `pthread_join()` function is used for this. It blocks the calling thread until the specified target thread terminates, allowing for a clean and synchronized completion of parallel tasks. Threads must be created as "joinable" to be used with this function. "Detached" threads, on the other hand, run independently and their resources are reclaimed automatically upon termination, but they cannot be joined.



🔗 Spiegazione

- **Joinable:** Il thread principale aspetta che il thread secondario finisca, e recupera il suo stato di uscita. Utile per sincronizzare e assicurarsi che un'operazione sia completata.
- **Detached:** Il thread principale non aspetta. Il thread secondario si "stacca" e le sue risorse vengono liberate automaticamente quando termina. Utile per operazioni indipendenti che non richiedono sincronizzazione.
- A **joinable** thread can become **detached** (not vice versa)
- Default attribute is **joinable** (not all implementation use default)
- **Detached** consume less resources

Thread Management:

Joining and Barriers are two distinct mechanisms for managing and synchronizing threads, each with a different purpose.

- **Joining:** A join operation occurs when one thread (typically the main thread) waits for another specific thread to terminate. *The calling thread (the one doing the join) blocks until the target thread has completed its execution.* This is useful when the calling thread needs the results or side effects of the target thread before it can proceed.
- **Barriers:** A barrier is a synchronization point where all threads in a group must arrive before any of them can proceed. No thread can pass the barrier until all threads have reached it. This is useful for ensuring that all threads have completed a certain phase of computation before moving on to the next phase.

```

C++
int pthread_barrier_init(pthread_barrier_t * barrier, pthread_barrierattr_t * attr,
    unsigned int count)
// count is the number of threads to be waited

int pthread_barrier_wait(pthread_barrier_t * barrier)
  
```

In summary:

- **Joining:** One thread waits for a specific thread to terminate.
- **Barriers:** All threads wait for all threads to reach a certain point.

2. Synchronization Primitives

Pthreads provides powerful tools to manage synchronization and prevent data races.

- **Mutexes (Mutual Exclusion):** A mutex is the most fundamental synchronization primitive. It acts as a lock that protects a "critical section" of code—a section that accesses shared data. Before entering the critical section, a thread must lock the mutex using `pthread_mutex_lock()`. If the mutex is already locked by another thread, the calling thread will block until it is released. After leaving the section, the thread unlocks it with `pthread_mutex_unlock()`. This ensures that only one thread can execute the critical section at a time. However, the improper use of multiple mutexes can lead to **deadlocks**, where two or more threads are stuck waiting for each other to release locks.
- **Condition Variables:** While mutexes are excellent for serializing access, they are not ideal for signaling between threads. For instance, a "consumer" thread might need to wait until a "producer" thread has added data to a queue. *Continuously locking a mutex to check if the condition is met (a technique called **polling**) is highly inefficient.* **Condition variables** solve this problem. A thread can wait on a condition variable, which atomically unlocks its associated mutex and puts the thread to sleep. It will only wake up when another thread *signals* the condition variable, indicating that the state it was waiting for has been achieved.

Spiegazione

- **Mutex (Mutual Exclusion):**
 - **Scopo principale:** Proteggere una sezione critica di codice o una risorsa condivisa, garantendo che **solo un thread alla volta** possa accedervi o modificarla.
 - **Funzionamento:** Agisce come un "lucchetto". Un thread lo blocca prima di entrare nella sezione critica e lo sblocca all'uscita. Se un altro thread tenta di bloccarlo mentre è già bloccato, quel thread si mette in attesa.
 - **Quando usarlo:** Per prevenire *data race* e garantire l'integrità dei dati condivisi.
- **Condition Variable:**
 - **Scopo principale:** Permettere ai thread di **attendere che una certa condizione sia vera** prima di procedere, e di essere notificati quando tale condizione si verifica.
 - **Funzionamento:** Viene sempre usata in combinazione con un mutex. Un thread che deve attendere una condizione sblocca il mutex e si mette in attesa sulla condition variable. Un altro thread, dopo aver modificato lo stato che rende vera la condizione, blocca il mutex e poi "segnala" la condition variable per risvegliare i thread in attesa.
 - **Quando usarlo:** Per la **sincronizzazione basata su eventi o stati**, ad esempio, un thread produttore che segnala a un thread consumatore che ci sono nuovi dati disponibili in una coda. Evita il *polling* (controllo continuo) inefficiente.

In sintesi: i **mutex** gestiscono l'accesso esclusivo a una risorsa, mentre le **condition variables** gestiscono la sincronizzazione dei thread in base a specifiche condizioni.

OpenMP: The High-Level Directive-Based Approach

OpenMP (Open Multi-Processing) is another API for multi-threaded, shared-memory programming, but it takes a much higher-level approach than Pthreads. Instead of a detailed library of functions, OpenMP is primarily based on **compiler directives** (in C/C++, these are `#pragma` statements).

This approach provides several key advantages:

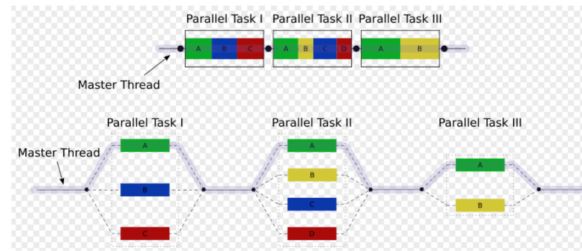
- **Ease of Use:** A few simple directives are often enough to parallelize significant portions of a program.
- **Portability:** OpenMP is a standard supported by most major compilers and works across a wide variety of platforms.
- **Incremental Parallelization:** A programmer can start with a serial program and introduce directives to parallelize the most computationally expensive parts (the "hotspots") without needing to restructure the entire application.

The Fork-Join Model

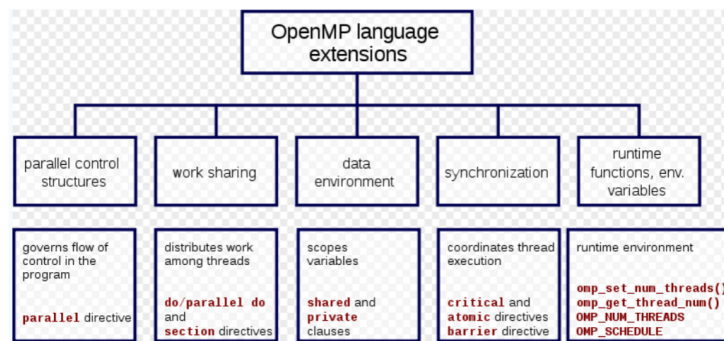
OpenMP is based on the **fork-join** paradigm. An OpenMP program begins execution as a single thread, called the **master thread**. When the master thread encounters a parallel directive:

1. **Fork:** It "forks" a team of **slave threads**.

2. **Parallel Execution:** The code within the parallel region is executed by all threads in the team concurrently.
3. **Join:** At the end of the parallel region, there is an implicit barrier. All threads synchronize, the slave threads are destroyed (or put to sleep), and only the master thread continues execution.



Key OpenMP Constructs



OpenMP's power comes from its rich set of directives and clauses that control parallelism, work distribution, and the data environment.

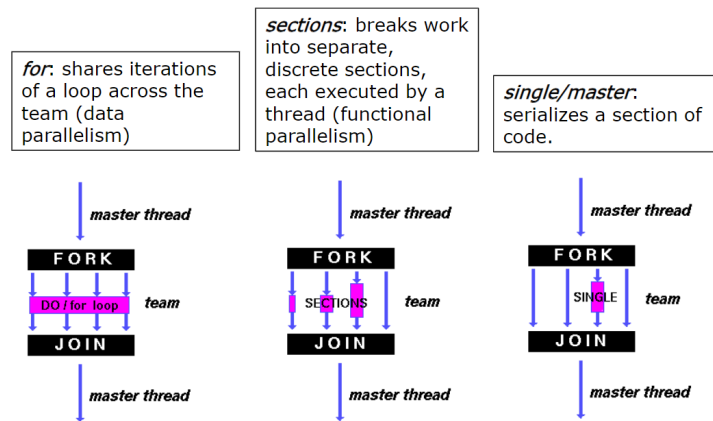
1. Parallel Control Structures

The fundamental directive is `#pragma omp parallel{}`. This defines a block of code that will be executed by a team of threads. The number of threads can be controlled via clauses like `num_threads(N)`, conditionally enabled with `if(condition)`, or set globally using runtime functions and environment variables. Then is possible to set the environment variable with `OMP_NUM_THREADS`

2. Work-sharing Constructs

These directives are used *inside* a parallel region to divide the work among the threads in the team. They do not create new threads, but rather assign the work within the enclosed code block.

- **#pragma omp for**: This is one of the most powerful constructs. It automatically divides the iterations of a **for** loop among the threads. This is a form of **data parallelism**.
 - The **schedule** clause can be used to control how the iterations are distributed
 1. **static** for pre-assigned chunks,
 2. **dynamic** for on-demand assignment, when a thread finishes one chunk, it is dynamically assigned another. The default chunk size is 1.
 - the **nowait** avoid sync after the end of the parallel loop
- **#pragma omp sections**: This divides work based on code blocks, not loop iterations. Each **#pragma omp section** within the **sections** block is executed once by one of the threads in the team. This is a form of **functional parallelism**.
- **#pragma omp single/master**: These directives ensure that the enclosed code block is executed by only one thread (**single**) or specifically by the master thread (**master**). This is useful for tasks like I/O or initialization that should only happen once.



- **#pragma omp task** → specifies a work unit which may be executed or deferred to another thread in the same team
- **#pragma omp taskwait** → introduces a barrier
- **#pragma omp taskyield** → interrupts execution of the current task... May be resumed by the same thread (tied clause) or by another (untied)

Costrutto	Descrizione	Tipo di Parallelismo	Caso d'Uso Tipico
#pragma omp for	Divide automaticamente le iterazioni di un ciclo for tra i thread del team.	Data Parallelism	Elaborazione di array, calcoli numerici su matrici, dove lo stesso compito si ripete su dati diversi.
#pragma omp sections	Assegna blocchi di codice (section) indipendenti a thread diversi. Ogni sezione viene eseguita una sola volta da un thread.	Functional Parallelism	Esecuzione di compiti funzionalmente diversi in parallelo (es. un thread calcola una FFT, un altro applica un filtro).
#pragma omp single / master	Assicura che un blocco di codice venga eseguito da un solo thread (single) o specificamente dal thread master (master).	Nessuno (Serializzazione)	Operazioni che devono essere eseguite una sola volta all'interno di una regione parallela, come inizializzazione o I/O.
#pragma omp task	Definisce un'unità di lavoro (task) che può essere eseguita immediatamente o posticipata per essere eseguita da un qualsiasi thread disponibile nel team.	Task Parallelism (Dinamico)	Algoritmi con parallelismo irregolare o ricorsivo, come l'esplorazione di alberi, grafi o liste concatenate.

3. Data Environment

Because OpenMP operates in a shared-memory model, most variables are **shared** by default. This means every thread can see and modify them, which can lead to data races. The programmer is responsible for managing the scope of variables using data scope attribute clauses:

- **private(list)**: *Each thread gets its own private copy of the variables in the list.* This is the primary way to avoid data races with loop counters or temporary variables.
 - a new object of the same type is declared once for each thread in the team
 - all references to the original object are replaced with references to the new object
 - assume that each variable is uninitialized for each thread – or use **firstprivate**
- **shared(list)**: *Explicitly declares that variables are shared among all threads.*
 - It exists in only one memory location and all threads can read or write to that address
 - It is the programmer's responsibility to ensure that multiple threads properly access shared variables
- **default(none)**: A good practice that forces the programmer to explicitly declare the scope of every variable, preventing accidental and hard-to-find bugs from default sharing.
 - **#pragma omp <name> default (shared | none)**
 - **default(shared)** is already the default, so it can be omitted
 - Specific variables can be exempted from the default using specific clauses (private, shared, etc.)

- Using none as a default requires that the programmer explicitly scope all variables
- **reduction(operator:list)**: A very *useful clause for parallel loops that perform aggregations* (like summing a list of numbers). It gives each thread a private copy of the reduction variable, performs the operation locally, and then safely combines the results from all threads into the global variable at the end of the loop.

4. Synchronization

OpenMP provides high-level directives for synchronization:

- **#pragma omp critical**: Defines a critical section, ensuring only one thread can execute it at a time (similar to a Pthreads mutex).
- **#pragma omp barrier**: Explicitly synchronizes all threads in the team. A thread will wait at the barrier until all other threads have reached it.
- **#pragma omp atomic**: Provides a lightweight way to protect a single statement (like **x++**) from race conditions, often by translating it to a single, atomic hardware instruction.

RUNTIME functions

- **int omp_get_num_threads()** → Returns the number of threads that are currently in the team executing the parallel region from which it is called
- **int omp_get_thread_num()** → Returns an identifier for the thread making this call. This number will be between 0 and **omp_get_num_threads - 1**. The master thread of the team is thread 0
- **void omp_set_num_threads(int num_threads)** → Sets the number of threads that will be used in the next parallel region
- **double omp_get_wtime()** → Provides a wall clock timing routine
- **double omp_get_wtick()** → Returns the number of seconds between two clock ticks

Environment variables are set to control execution of all programs, for example with the export command in bash

```
❑ OMP_SCHEDULE
❑ OMP_NUM_THREADS
❑ OMP_NESTED
❑ OMP_STACKSIZE
❑ OMP_WAIT_POLICY
❑ OMP_PROC_BIND
```

D. Parallel Computing: From Dependencies to Design Patterns

The Foundation: Parallelism and Dependence

Understanding the Constraints of Parallel Execution

Parallel computing aims to execute multiple operations simultaneously to accelerate computation. However, this is not always possible. *The ability to execute code in parallel is fundamentally constrained by dependencies.* A dependency exists when one operation relies on the completion and result of a previous operation. To achieve a correct outcome, these dependencies must be respected, which imposes a specific sequence on the operations and limits parallelism.

Parallel execution must address three types of dependencies:

1. **Control Dependencies:** The flow of the program dictates the order (e.g., an **if** statement).
2. **Data Dependencies:** One operation needs the data produced by another.
3. **System Dependencies:** Operations require the same hardware resource.

For two statements to be truly independent and executable in any order without changing the final result, their order of execution must not matter. If the order *does* affect the outcome, the statements are considered **dependent**.

The Three Core Data Dependencies

Data dependencies are the most common and critical constraints in parallel programming. *They are categorized based on how statements read from and write to memory locations (variables).*

True (Flow) Dependence: The Read-After-Write (RAW) Constraint

This is the most intuitive type of dependency. A statement **S2** has a true dependence on **S1** if **S2** reads a value that was written by **S1**. This creates a fundamental flow of data that must be preserved.

Example

```
S1: a = 1;  
S2: b = a; // S2 must wait for S1 to write to 'a'
```

Here, **S2** cannot execute before **S1**. This is a "true" dependency that cannot be eliminated without changing the algorithm's logic.

Anti-Dependence: The Write-After-Read (WAR) Hazard

An anti-dependence occurs when a statement **S2** writes to a memory location that a preceding statement **S1** reads. The order must be preserved to ensure **S1** reads the *old* value before **S2** overwrites it.

Example

```
S1: a = b; // S1 needs the original value of 'b'  
S2: b = 1; // S2 overwrites 'b'
```

If **S2** executed before **S1**, **S1** would incorrectly read the value **1** for **b**.

Output Dependence: The Write-After-Write (WAW) Hazard

An output dependence exists when two statements, **S1** and **S2**, both write to the same memory location. The order of execution is crucial to ensure that the final value in that location is the one from the logically last statement.

Example

```
S1: a = f(x); // S1 writes to 'a'  
S2: a = b;    // S2 also writes to 'a'
```

The final value of **a** must be the value of **b**, as dictated by the program order.

Taming Dependencies: Elimination and Identification

While true (flow) dependencies are inherent to the algorithm, anti- and output dependencies are often called **"name dependencies"**.

They exist only because the same variable name (**a**, **b**, etc.) was reused.

These can frequently be eliminated by renaming variables, which breaks the dependency and unlocks more parallelism.

To formally identify these dependencies, we can analyze the **IN** and **OUT** sets for each statement:

- IN(S)**: The set of variables read (used) by statement **S**.

- **OUT(S)**: The set of variables written (modified) by statement **S**.

Given two statements **S1** and **S2**, a dependency exists if:

- **OUT(S1) ∩ IN(S2) ≠ ∅** → **Flow Dependence**
- **IN(S1) ∩ OUT(S2) ≠ ∅** → **Anti-Dependence**
- **OUT(S1) ∩ OUT(S2) ≠ ∅** → **Output Dependence**

Unleashing Parallelism in Loops

Loops are the most significant source of parallelism in many applications. Analyzing dependencies within loops is critical to parallelizing them effectively.

Loop-Level Parallelism: The Primary Source of Concurrency

When iterations of a loop are independent of each other, they can be executed in parallel. This is often called a **DOALL** loop (or **foreach**). However, dependencies can exist *between* iterations, preventing this.

Distinguishing *Loop-Carried* and *Loop-Independent* Dependencies

- **Loop-Independent Dependence:** A dependency that exists within the *same* iteration of a loop. The statements within a single iteration have a dependency, but different iterations are independent.
- **Loop-Carried Dependence:** A dependency that crosses iteration boundaries. An operation in one iteration depends on the result of an operation in a *previous* iteration.
- **Example (Loop-Carried Dependence):**

```
for (i=1; i<100; i++) {
    a[i] = a[i-1] + 100;
    // Each iteration depends on the previous one
}
```

This loop cannot be fully parallelized because of the true (flow) loop-carried dependency. Iteration **i** needs the value **a[i-1]** calculated in iteration **i-1**.

Strategies for Nested Loops

In nested (annidato) loops, a dependency might be carried by the inner loop or the outer loop.

- If a dependency is carried by the **inner loop**, the inner loop cannot be parallelized, but the outer loop might be.
- If a dependency is carried by the **outer loop**, the inner loop can often be fully parallelized.

Examples

1. Dipendenza nel ciclo interno (Inner Loop)

In questo caso, il calcolo in un'iterazione del ciclo interno dipende dall'iterazione precedente **dello stesso ciclo interno**. Di conseguenza, solo il ciclo esterno può essere parallelizzato.

```
// Il ciclo esterno (su 'i') può essere parallelizzato perché ogni riga è
// indipendente.
#pragma omp parallel for
for (int i = 0; i < N; i++) {

    // Il ciclo interno (su 'j') NON può essere parallelizzato
    // perché a[i][j] dipende da a[i][j-1] (stessa riga, colonna precedente).
    for (int j = 1; j < M; j++) {
        a[i][j] = a[i][j-1] * 2;
    }
}
```

Spiegazione: Ogni riga `i` della matrice `a` viene calcolata in modo indipendente dalle altre. Tuttavia, all'interno di una singola riga, ogni elemento `a[i][j]` dipende dal valore del suo vicino a sinistra `a[i][j-1]`. Questa è una dipendenza portata dal ciclo interno, che quindi deve rimanere sequenziale.

2. Dipendenza nel ciclo esterno (Outer Loop)

Qui, il calcolo in un'iterazione del ciclo esterno dipende dall'iterazione precedente **del ciclo esterno**. Questo permette di parallelizzare il ciclo interno.

```
// Il ciclo esterno (su 'i') NON può essere parallelizzato
// perché la riga 'i' dipende interamente dalla riga precedente 'i-1'.
for (int i = 1; i < N; i++) {

    // Il ciclo interno (su 'j') PUÒ essere parallelizzato
    // perché ogni calcolo a[i][j] è indipendente dagli altri nella stessa riga.
    #pragma omp parallel for
    for (int j = 0; j < M; j++) {
        a[i][j] = a[i-1][j] + 10;
    }
}
```

Spiegazione: Per calcolare qualsiasi elemento della riga **i**, è necessario il valore corrispondente dalla riga **i-1**. Questa dipendenza è portata dal ciclo esterno, che deve quindi essere eseguito in sequenza. Tuttavia, una volta fissata una riga **i**, il calcolo di ogni **a[i][j]** è indipendente da tutti gli altri elementi **a[i][k]** sulla stessa riga. Pertanto, il ciclo interno può essere eseguito in parallelo.

When Dependencies Persist: The Role of Synchronization

If a dependency is fundamental to the algorithm and cannot be eliminated, **synchronization mechanisms** must be used. These mechanisms (like locks, barriers, or semaphores) enforce the correct ordering of events across different processors, preserving correctness at the cost of some parallel efficiency.

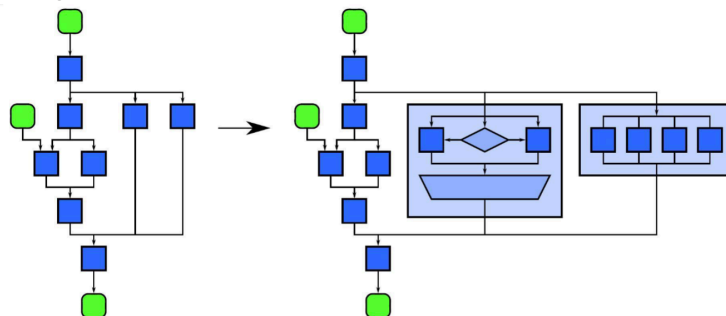
A Vocabulary for Parallel Design: Algorithmic Patterns

Parallel patterns provide a high-level "vocabulary" for designing and discussing parallel algorithms. They are recurring combinations of task distribution and data access that solve common problems.

Core Structural Pattern: Nesting

Nesting is a compositional pattern that allows patterns to be hierarchically composed.

Any task block within a pattern diagram can be replaced by another, more complex pattern, allowing for the construction of sophisticated algorithms.



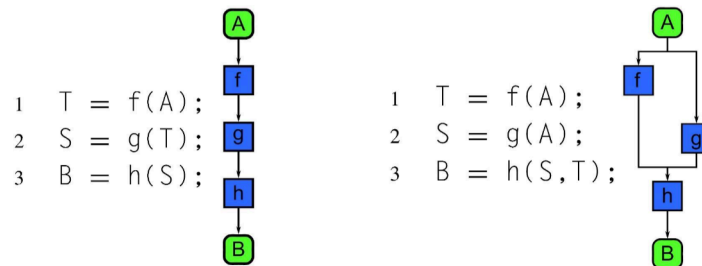
Serial Control Patterns

The Serial Control Patterns form the foundation of structured serial programming. It is crucial to understand these patterns, as they are the basis for serial algorithms that programmers often seek to parallelize.

The structured serial programming paradigm is based on four fundamental serial control patterns:

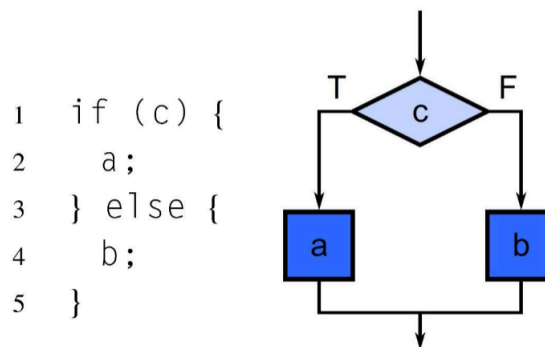
1. Sequence:

- This is an **ordered list of tasks** that must be executed in a specific order.
- The assumption inherent in this pattern is that the program text ordering will be followed.



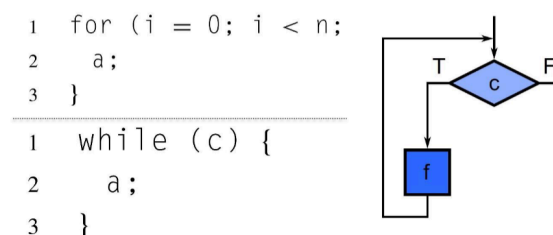
2. Selection:

- In this pattern, a **condition (c)** is evaluated first.
- Based on the true or false result of **c**, either task **a** or task **b** is executed.
- The core assumptions are that tasks **a** and **b** are never executed before **c**, and that only one of them (either **a** or **b**) is executed, never both.



3. Iteration:

- This pattern involves evaluating a condition (**c**). If **c** is true, task **a** is evaluated, and the condition **c** is then evaluated again, repeating until **c** becomes false.
- A common complication arising when attempting to parallelize iteration is the **potential for dependencies** to exist between previous iterations.



4. Recursion:

- Recursion is described as a **dynamic form of nesting** that allows functions to call themselves.
- **Tail recursion** is a special type of recursion that can be converted into iteration, which is particularly important in functional languages.

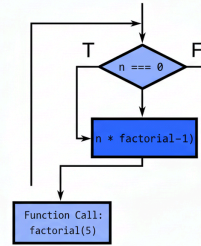
RECURSION

Function Definition:

```
1 function factorial(n) { {
2   if (n
3     if (n == 0) {
4       return 1;
5     }
6   }
```

Function Call

```
1 () else {
2   return n * factorial(-1- -1);
3 }
4 result = factorial(5);
```

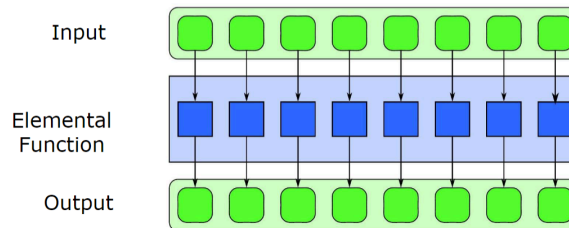


These four patterns can be hierarchically composed using the **Nesting Pattern**.

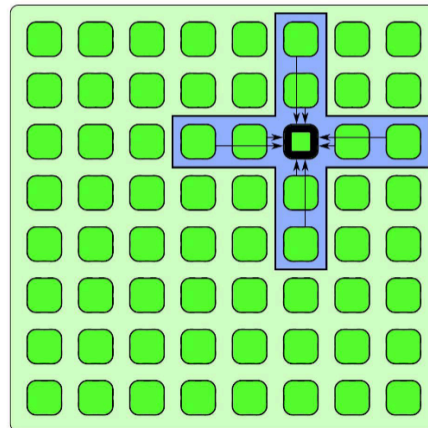
Parallel Control Patterns: Extending Serial Logic

These patterns are parallel counterparts to serial control structures like sequence, selection, and iteration.

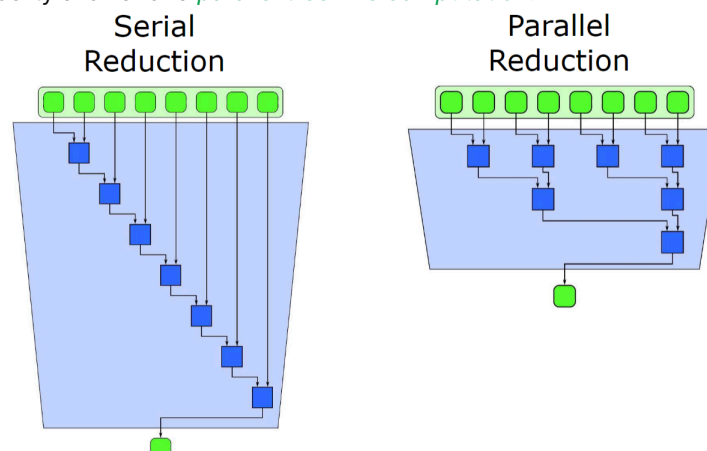
- **Fork-Join:** A master thread *forks off multiple parallel tasks and then waits (joins)* for them to complete before continuing.
- **Map:** *Applies a function to every element of a collection*. If the function is pure (no side effects or dependencies between elements), this pattern is **embarrassingly parallel**.



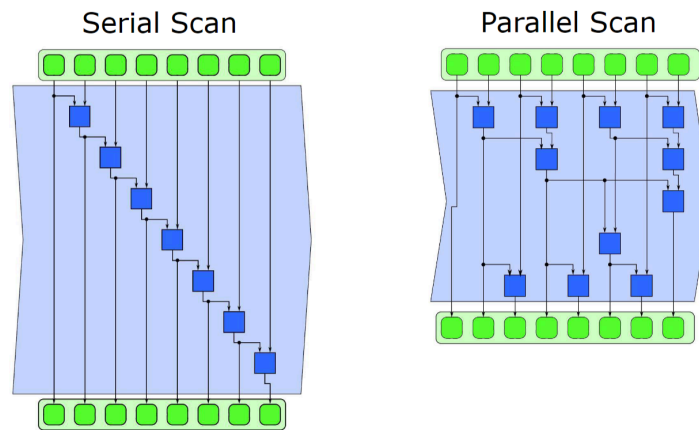
- **Stencil:** A generalization of Map where the function for each element *accesses a "neighborhood" of adjacent elements*. This is common in *image processing* and scientific simulations.



- **Reduction:** Combines *all elements in a collection into a single result* using an associative operator (e.g., sum, max, min). The associative property allows for a *parallel tree-like computation*.



- **Scan:** Also known as a *"prefix sum,"* it computes all partial reductions of a collection. For each output element, it computes the reduction of all input elements up to that point. It can be parallelized but is *more complex than reduction*.



- **Recurrence:** A pattern similar to a serial loop with loop-carried dependencies, where *each step depends on the previous one.*

Parallel Data Management Patterns: Organizing and Moving Data

These patterns deal with how data is structured and shared among parallel workers.

The four main patterns of **serial** data management are:

1. Random Read and Write

- Memory locations are indexed using addresses.
- Pointers are typically used to reference these memory addresses.
- A significant issue arises from **aliasing**, which is the uncertainty of whether two pointers refer to the same object, especially when serial code is later parallelized.

2. Stack Allocation

- This pattern is useful for dynamically allocating data in a **LIFO (Last-In, First-Out) manner**.
- It is highly **efficient** because an arbitrary amount of data can be allocated in constant time.
- Stack allocation also preserves **locality**.
- When an algorithm using stack allocation is parallelized, typically each thread receives its own stack, thereby preserving thread locality.

3. Heap Allocation

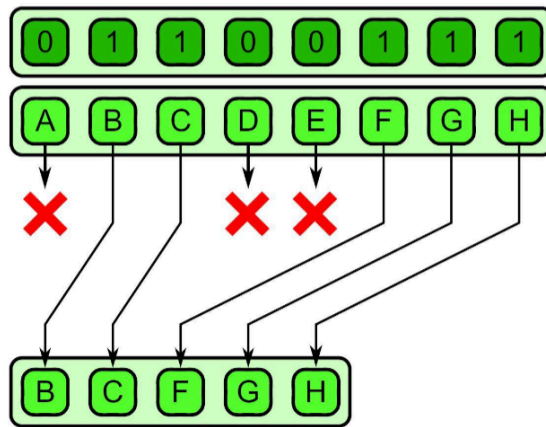
- This technique is necessary when data cannot be allocated in a LIFO fashion.
- In contrast to stack allocation, heap allocation is generally **slower and more complex**.
- When dynamically allocating memory in parallel, a **parallelized heap allocator** should be used; this type of allocator keeps separate pools for each parallel worker.

4. Objects

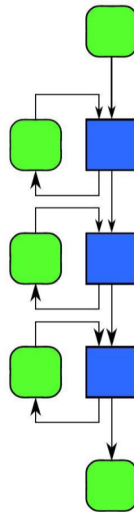
- Objects are language constructs designed to **associate data with the code** necessary to manipulate and manage that data.
- Objects can include member functions and are considered members of a class.
- Parallel programming models tend to generalize objects in various ways.

Then the main patterns of **parallel** data management are:

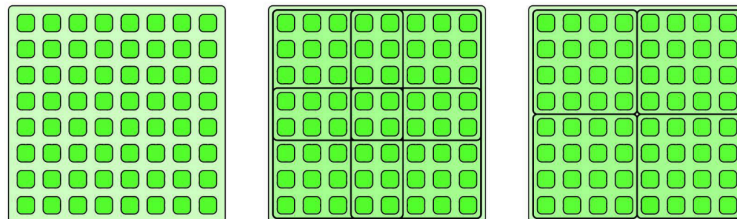
1. **Pack:** Filters a collection to eliminate unused elements, creating a new, compact collection.



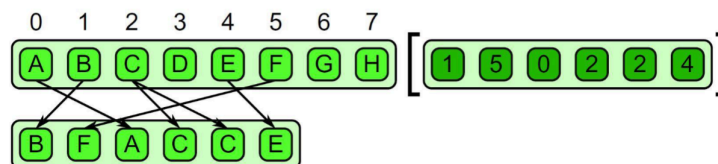
2. **Pipeline:** Creates a chain of producer-consumer tasks, where *the output of one stage becomes the input for the next*. Parallelism is achieved by having multiple data items in the pipeline simultaneously.



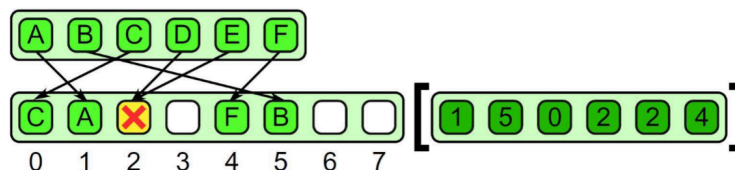
3. **Geometric Decomposition:** Divides a data structure (often an array or grid) into smaller sub-collections that can be processed in parallel.



4. **Gather:** An irregular read operation that creates a new collection by reading elements from an input collection at indices specified by a third collection.



5. **Scatter:** The inverse of Gather. It is an irregular write operation that writes elements from an input collection into an output collection at specified indices. It can lead to **race conditions** if multiple writes target the same location.





Other Pattern

- **Superscalar Sequences:** This pattern involves writing a sequence of tasks where the execution order is determined **only by dependencies** between the tasks. (This is analogous to how a superscalar processor executes instructions out-of-order when dependencies permit.)
- **Futures:** This pattern is conceptually **similar to the fork-join pattern**, but a key distinction is that the tasks defined by **Futures do not need to be nested hierarchically**.

These patterns exist alongside other advanced control schemes like:

- **Speculative Selection:** A generalization of serial selection where the condition and both potential outcomes can be executed in parallel.
- **Workpile:** A variation of the map pattern where each instance of the elemental function can generate and add more instances to the collective work queue, terminating when no more work is available.
- **Search:** Used to find data within a collection that meets specific criteria.
- **Segmentation:** Operations performed on non-overlapping, non-uniformly sized partitions of 1D collections.
- **Expand:** A combination of the pack and map patterns.
- **Category Reduction:** Finding all elements with the same label within a collection and reducing them.

A Deep Dive into the Map Pattern

Map: The Essence of Embarrassingly Parallel Computation

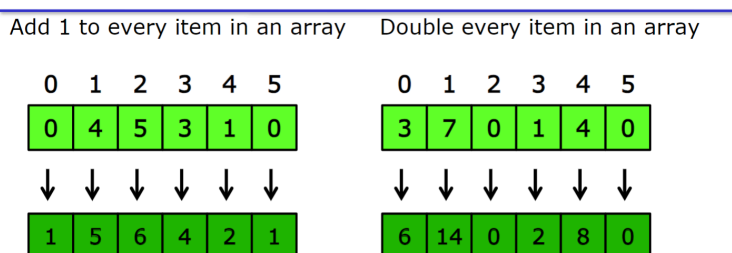
The Map pattern is arguably the most important and common parallel pattern. It embodies the idea of "doing the same thing to many pieces of data." It corresponds to a **foreach** loop where every iteration is completely **independent** of all others.

This independence allows the work to be distributed across any number of processors with minimal coordination, leading to significant speedups.

The Cardinal Rule: Independence and Pure Functions

The key to unlocking the power of the Map pattern is ensuring perfect independence. The function applied to each element should be **"pure"** (or "pure-ish"), meaning it should not modify any shared state. Violating this rule by introducing shared, mutable state can lead to severe concurrency bugs:

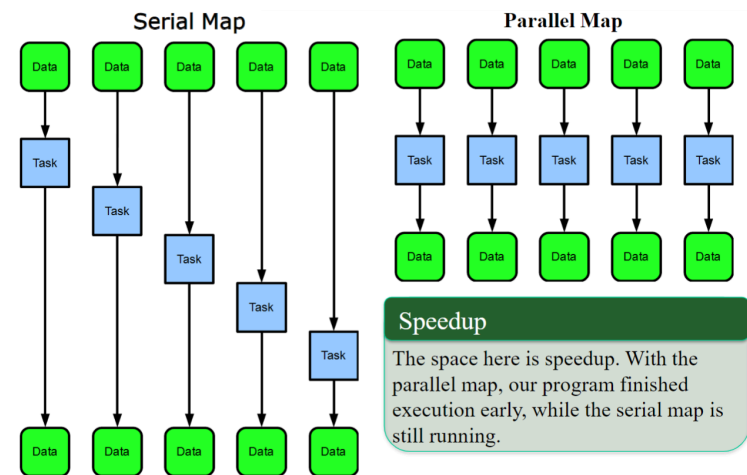
- **Non-determinism:** The program produces different results on different runs.
- **Data Races:** Multiple threads access the same memory location without synchronization, and at least one access is a write.
- **Undefined Behavior:** The program may crash or produce nonsensical results.



Key Point: An operation is a map if it can be applied to each element without knowledge of neighbors.

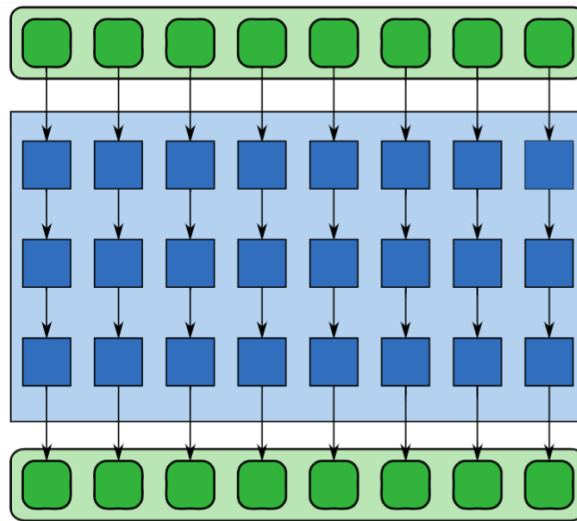
Optimizing Sequences of Maps

In many applications, especially vector math, several map operations occur in sequence. A naïve implementation would execute each map one after another, writing the entire intermediate result to memory at each step. This is highly inefficient due to low arithmetic intensity and high memory bandwidth usage.



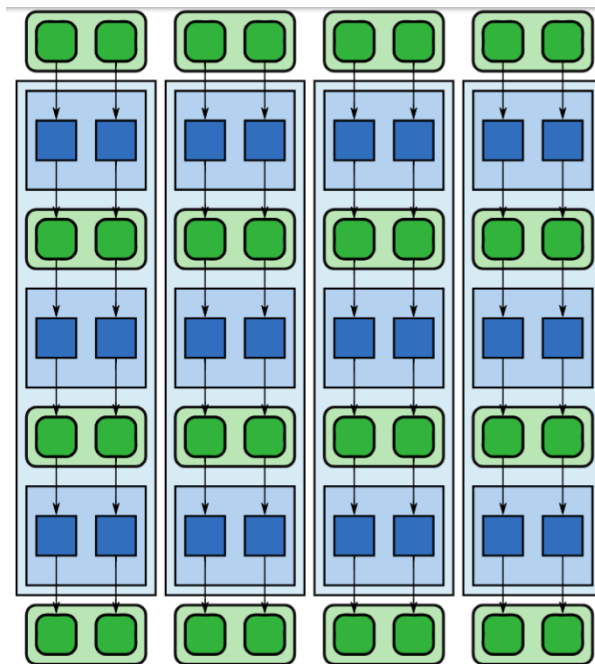
Solution 1: Code Fusion for Maximum Efficiency

Code Fusion (or Loop Fusion) combines the operations from multiple sequential maps into a single, more complex map. This allows intermediate results to be kept in fast CPU registers instead of being written to and read from main memory, dramatically reducing the memory bottleneck and improving performance.



Solution 2: Cache Fusion for Improved Locality

When full code fusion is impractical, **Cache Fusion** (or Tiling) offers an alternative. The data is broken into smaller blocks, or "tiles," that fit comfortably into a CPU's cache. Each processor takes a tile and performs the *entire sequence* of map operations on it before moving to the next tile. This ensures that the data for the sequence of operations remains in the fast cache, avoiding repeated trips to main memory.



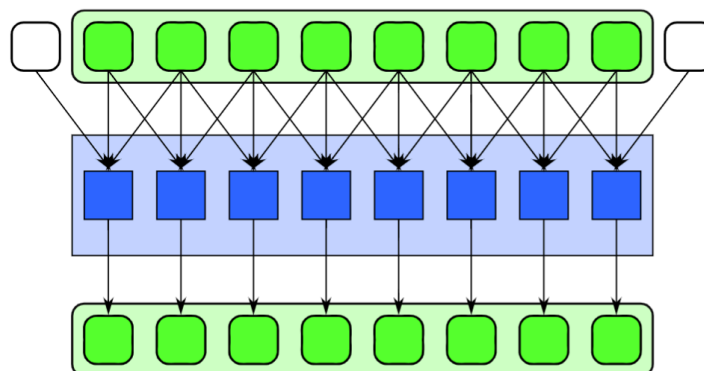
Related Patterns

These patterns are classified as **Related Patterns** to the Map Pattern or as fundamental algorithmic strategies in parallel computation. They provide specialized structures for decomposing and executing work:

1. Stencil

The **Stencil** pattern is a **generalization of the Map pattern**.

- **Definition:** In a Stencil operation, the elemental function accesses a set of "**neighbors**" of its input, offset from the element's usual position.
- **Application:** Stencil operations are common in fields like imaging and solving Partial Differential Equations (PDEs). It is often combined with the **Iteration** pattern to evolve a system through time, such as in iterative solvers.
- **Example:** The Jacobi Iteration Method for the Laplace Equation utilizes a Stencil pattern. This method iteratively computes a new solution by replacing every interior grid point with the average of its four neighbors from the previous iteration, while keeping the boundary values fixed (Dirichlet boundary conditions).
- **Data Reuse:** Boundary conditions must be handled carefully in this pattern. The canonical term "**halo**" is sometimes used to refer to the input data that continuously overlaps and is reused among threads for subsequent calculations in stencil kernels.



2. Workpile

The **Workpile** pattern is a **generalization of the Map pattern** that handles dynamically generated tasks.

- **Mechanism:** Work items can be **added to the map while it is in progress**, originating from inside the elemental function instances themselves.
- **Execution Flow:** Work grows and is consumed by the map.
- **Termination:** The Workpile pattern terminates only when **no more work is available**.

3. Divide-and-Conquer (D&C)

The **Divide-and-Conquer** pattern is a highly scalable algorithmic approach.

- **Definition:** It applies when a problem can be **repeatedly subdivided into identical subproblems** recursively until a base case is reached that can be solved serially. The solutions to the base cases are then combined to solve the original problem.
- **Implementation:** Subdivision can be performed either **iteratively or through recursive calls**.
- **Parallelization:** Although typical of sequential algorithms, D&C is easily parallelized by assigning **each subproblem to a different processor**.
- **Complexity:** The complexity of D&C algorithms is often written using recurrence relations in the form $T(n) = kT(n/b) + O(n)$, where k is the number of sub-problems and $O(n)$ is the cost of recombination plus the base case solution. Examples include Matrix Multiplication and Merge Sort.

Case Study: SAXPY (Scaled Vector Addition)

The SAXPY Algorithm: A Classic Map Problem

SAXPY is a fundamental operation from the BLAS (Basic Linear Algebra Subprograms) library that computes $y = a * x + y$, where x and y are vectors and a is a scalar. This operation scales the vector x by a and adds the result to vector y .

This is a perfect example of a Map pattern. The calculation for each element $y[i]$ is completely independent of the calculation for any other element $y[j]$.

	0	1	2	3	4	5	6	7	8	9	10	11
a	4	4	4	4	4	4	4	4	4	4	4	4
* x	2	4	2	1	8	3	9	5	5	1	2	1
+												
y	3	7	0	1	4	0	0	4	5	3	1	0
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
y	11	23	8	5	36	12	36	49	50	7	9	4

→ You can use 12 processors or 6 processors or 2 processors

A Comparative Look at Implementations

The SAXPY algorithm can be implemented using various parallel programming models:

- **Serial:** A simple **for** loop.

```
1 void saxpy_serial(  
2     size_t n,           // the number of elements in the vectors  
3     float a,            // scale factor  
4     const float x[],    // the first input vector  
5     float y[]           // the output vector and second input vector  
6 ) {  
7     for (size_t i = 0; i < n; ++i)  
8         y[i] = a * x[i] + y[i];  
9 }
```

- **TBB (Threading Building Blocks):** Uses a **parallel_for** with a lambda function to express the loop body.

```

1 void saxpy_tbb(
2     int n,        // the number of elements in the vectors
3     float a,      // scale factor
4     float x[],    // the first input vector
5     float y[]     // the output vector and second input vector
6 ) {
7     tbb::parallel_for(
8         tbb::blocked_range<int>(0, n),
9         [&](tbb::blocked_range<int> r) {
10             for (size_t i = r.begin(); i != r.end(); ++i)
11                 y[i] = a * x[i] + y[i];
12         }
13     );
14 }

```

- **Cilk Plus:** Provides a very concise `cilk_for` keyword to parallelize the loop.

```

1 void saxpy_cilk(
2     int n,        // the number of elements in the vectors
3     float a,      // scale factor
4     float x[],    // the first input vector
5     float y[]     // the output vector and second input vector
6 ) {
7     cilk_for (int i = 0; i < n; ++i)
8         y[i] = a * x[i] + y[i];
9 }

```

- **OpenCL:** Requires writing a "kernel" function that executes on a GPU, with boilerplate code to manage memory transfers.

```

__kernel void
saxpy_opengl(
    __constant float a,
    __global float* x,
    __global float* y
) {
    int i = get_global_id(0);
    y[i] = a * x[i] + y[i];
}

```

- **OpenMP:** Uses a simple `#pragma omp parallel for` directive just before the `for` loop to instruct the compiler to parallelize it.

```

1 void saxpy_omp(
2     int n,        // the number of elements in the vectors
3     float a,      // scale factor
4     float x[],    // the first input vector
5     float y[]     // the output vector and second input vector
6 ) {
7     #pragma omp parallel for
8     for (int i = 0; i < n; ++i)
9         y[i] = a * x[i] + y[i];
10 }

```

Performance in the Real World

A performance graph of an OpenMP SAXPY implementation shows a common real-world behavior. As the number of threads increases from 1, the execution time decreases significantly, demonstrating the benefit of parallelism.

However, after a certain point (e.g., 4 threads on a machine with 4 physical cores), the performance may level off or even degrade. This is because the benefits of adding more threads are outweighed by factors such as:

- **Overhead:** The cost of creating and managing threads.

- **Memory Bandwidth Saturation:** The parallel processors consume data from memory faster than it can be supplied.
- **Limited Physical Cores:** Using more threads than physical cores (hyper-threading) may not always improve performance for memory-bound tasks.

Spiegazioni

In OpenMP, `proc_bind` è una clausola che controlla l'affinità dei thread, ovvero come i thread vengono "legati" ai processori fisici o logici (core, socket) del sistema. Questo può avere un impatto significativo sulle prestazioni, specialmente in sistemi multi-core o multi-socket, a causa della gestione della cache e della latenza di accesso alla memoria.

Ecco cosa fanno `proc_bind(spread)` e `proc_bind(close)`:

1. `proc_bind(spread)`:

- **Cosa fa:** Tenta di distribuire i thread del team OpenMP il più ampiamente possibile tra le unità di elaborazione disponibili. L'obiettivo è massimizzare la distanza tra i thread, assegnandoli a core diversi, possibilmente su socket diversi, per ridurre la contesa per le risorse della cache di livello inferiore (L1, L2) e per i core stessi.
- **Quando è utile:**
 - Quando i thread eseguono calcoli indipendenti e intensivi che beneficiano di avere la propria cache dedicata.
 - Per ridurre il "false sharing" (falsa condivisione) se i thread accedono a dati vicini in memoria ma non condivisi, ma che finiscono nella stessa linea di cache.
 - In scenari dove la larghezza di banda della memoria è un fattore limitante e distribuire i thread su più controller di memoria (tipico dei sistemi multi-socket) può aiutare.
- **Potenziali vantaggi:** Aumenta la probabilità che i thread siano su socket diversi, il che può aumentare la latenza di comunicazione se i thread devono scambiare dati frequentemente (accesso alla memoria non locale, NUMA).

2. `proc_bind(close)`:

- **Cosa fa:** Tenta di raggruppare i thread del team OpenMP il più vicino possibile tra loro sulle unità di elaborazione. L'obiettivo è mantenere i thread sullo stesso core (se supportato da hardware come Hyper-Threading/SMT), sullo stesso core fisico, o sullo stesso socket, per sfruttare la condivisione della cache di livello superiore (L2, L3) e ridurre la latenza di comunicazione.
- **Quando è utile:**
 - Quando i thread *condividono frequentemente dati e beneficiano di avere quei dati nella stessa cache* (ad esempio, L2 o L3 condivisa).
 - Per ridurre la latenza di accesso alla memoria, mantenendo i thread sullo stesso socket e quindi accedendo alla memoria locale.
 - In scenari dove la comunicazione tra thread è intensa.
- **Potenziali svantaggi:** Aumenta la contesa per le risorse della cache di livello inferiore (L1) e per le unità di esecuzione del core, il che può portare a un rallentamento se i thread sono troppo "vicini" e non c'è abbastanza capacità hardware per gestirli efficientemente.

In sintesi:

- `spread` cerca di massimizzare l'isolamento dei thread per ridurre la contesa delle risorse e sfruttare al meglio le cache individuali.
- `close` cerca di massimizzare la vicinanza dei thread per sfruttare le cache condivise e ridurre la latenza di comunicazione.

La scelta tra `spread` e `close` (o il default `master` o `true`) dipende molto dalla natura del carico di lavoro, dalla topologia hardware del sistema e dal modo in cui i dati vengono acceduti e condivisi tra i thread. Spesso è necessario sperimentare per trovare l'opzione migliore per una specifica applicazione.

1. `grainsize` clause:

- **Definizione:** Questa clausola definisce **quante iterazioni** di un ciclo dovrebbero essere eseguite da una singola task.
- **Flessibilità:** È interessante notare che lo standard OpenMP permette a `grainsize` di essere un **intervallo** (ad esempio, da 8 a 16 iterazioni, come nel tuo esempio). Questo offre all'implementazione OpenMP una certa flessibilità nella scelta del numero esatto di iterazioni per task (e di conseguenza, nel numero totale di task create). L'implementazione può scegliere un valore all'interno dell'intervallo che ritiene più efficiente in base al carico di lavoro e alle risorse disponibili.
- **Scopo:** L'obiettivo è bilanciare l'overhead di creazione e gestione delle task con la granularità del lavoro. Un `grainsize` troppo piccolo può generare troppe task e aumentare l'overhead; un `grainsize` troppo grande può ridurre il parallelismo disponibile.

2. `num_tasks` clause:

- **Definizione:** Questa clausola specifica **esattamente quante task** dovrebbero essere create per un dato costrutto.
- **Aggiustamento della dimensione del chunk:** A differenza di `grainsize` che definisce la dimensione delle task, `num_tasks` definisce il numero totale di task. Di conseguenza, l'implementazione OpenMP **regola la dimensione del "chunk"** (cioè, il numero di iterazioni per task) in modo che il numero totale di task create corrisponda a quello specificato da `num_tasks`.
- **Scopo:** Questa clausola è utile quando si ha un'idea precisa di quante task si desidera generare, magari per ottimizzare l'utilizzo di un certo numero di core o per allinearsi a una specifica strategia di bilanciamento del carico.

In sintesi:

- `grainsize` ti permette di dire: "Ogni task dovrebbe gestire circa X iterazioni (o un intervallo tra X e Y)". Il numero totale di task sarà una conseguenza di questa scelta.
- `num_tasks` ti permette di dire: "Voglio esattamente N task". La dimensione di ogni task sarà una conseguenza di questa scelta.

Entrambe le clausole offrono meccanismi per controllare la granularità del parallelismo basato su task, permettendo agli sviluppatori di ottimizzare le prestazioni in base alle caratteristiche del loro algoritmo e dell'hardware.

#BOOKMARK-Study

E. Parallel Computing: Collectives, Reduce, and Scan

Introduction to Collective Operations

In parallel programming, a **collective operation** is one that involves a group of processes or threads working together on a shared collection of data. The goal is to coordinate their efforts to produce a combined result. This is *distinct from embarrassingly parallel problems where each thread can work in complete isolation*.

While there are several collective patterns, including Partition, Scatter, and Gather, this exploration will focus on two foundational ones:

- **Reduce:** A pattern that combines all elements in a collection into a single, summary value.
- **Scan:** A pattern that computes a running summary for all elements, producing a new collection of partial results.

The Reduce Pattern

Core Concept

The **Reduce** pattern is used to distill a large collection of data down to a single summary value.

This is achieved by repeatedly applying a **combiner function** to pairs of elements until only one value remains.

For a reduction to be effectively parallelized, the combiner function must be **associative**. This means that the order in which elements are grouped for the operation does not change the final result. For example, with addition: $(a + b) + c$ is the same as $a + (b + c)$.

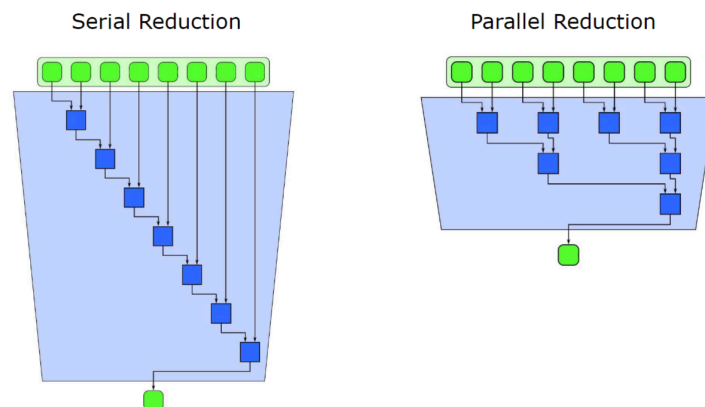
Common associative functions used for reduction include:

- **Addition:** To find the sum of all elements.
- **Multiplication:** To find the product of all elements.
- **Maximum / Minimum:** To find the largest or smallest value in the collection.

Parallelizing the Reduction

A reduction can be performed serially or in parallel, with significant performance differences.

- **Serial Reduction:** This is a sequential process. It starts with the first two elements, combines them, then combines that result with the third element, and so on, until the end of the collection is reached. This is inherently slow as it can only perform one operation at a time.
- **Parallel Reduction:** This approach restructures the problem into a **tree-like hierarchy**. It begins by combining many pairs of elements simultaneously in the first step. The results of that step are then combined in pairs, and this process continues up the tree until a single final value is produced. This is dramatically faster because the number of steps grows logarithmically with the size of the input, not linearly.



(Image illustrating the linear flow of serial reduction versus the tree-like structure of parallel reduction)

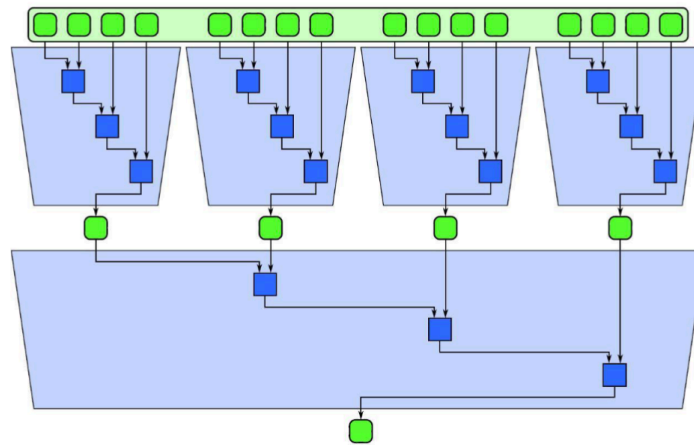
For example, to sum the list $[1, 2, 5, 4, 9, 7, 0, 1]$ in parallel:

1. **Step 1:** Compute $(1+2)$, $(5+4)$, $(9+7)$, and $(0+1)$ simultaneously to get $[3, 9, 16, 1]$.
2. **Step 2:** Compute $(3+9)$ and $(16+1)$ simultaneously to get $[12, 17]$.
3. **Step 3:** Compute $(12+17)$ to get the final result, 29 .

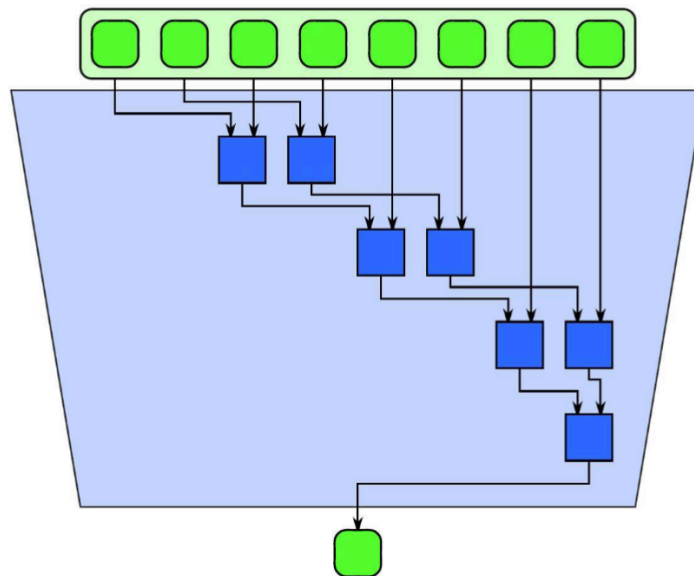
This takes only three steps, whereas a serial reduction would have taken seven.

Optimization Strategies

- **Tiling:** For very large datasets, a technique called **tiling** is used. The data is broken into large, contiguous chunks (tiles). Each available worker or thread performs a fast **serial** reduction on its assigned tile. Once all workers have finished, a final parallel reduction is performed on the intermediate results from each tile. This strategy optimizes memory access and balances the workload efficiently.



- **Vectorization:** Modern CPUs can perform a single instruction on multiple pieces of data at once (SIMD). Vectorization is a technique that leverages this by structuring the reduction to operate on small vectors of data simultaneously, further increasing performance at a low level.



Fusing Map and Reduce

The Reduce pattern can be combined with a **Map** pattern. A map is an operation that applies a function to every element in a collection, producing a new collection of the same size. By "fusing" them, we first *map* a function over the input and then immediately *reduce* the results. This is the foundation of the famous **MapReduce** programming model.

Practical Example: The Dot Product

A classic example of a **MapReduce** operation is the **dot product** of two vectors. The dot product is essential in physics, graphics, and video games for calculating projections and angles between vectors.

To compute the dot product of vectors **a** and **b**:

1. **Map:** Multiply the corresponding components of the vectors together ($a[i] * b[i]$).
2. **Reduce:** Sum all the products from the map step using addition as the combiner.

A gaming analogy helps illustrate this: In a racing game like Mario Kart, your speed boost depends on how well your kart's direction aligns with the direction of a boost pad. The dot product calculates this alignment. A perfect alignment (red and blue vectors pointing the same way) gives maximum boost, while driving perpendicularly across it gives zero boost.

A Note on Precision

When using floating-point numbers (e.g., **float**, **double**), precision can be a concern. Floating-point arithmetic is not perfectly associative due to rounding. This means that a parallel reduction, which reorders the operations, can

produce a slightly different result than a serial reduction. This is often an acceptable trade-off for the massive gain in speed, but it is a critical factor to be aware of in scientific and financial applications.

The Scan Pattern

Core Concept

The **Scan** pattern, also known as a **prefix sum**, is a more complex collective operation. Instead of producing a single summary value like Reduce, it produces a new sequence of the same length as the input. Each element in the output sequence is the result of a reduction of all the elements that came *before* it in the input.

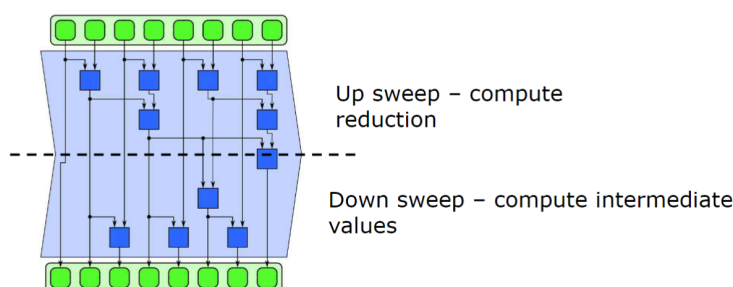
Scan is trickier to parallelize than Reduce because each output element depends on the previous ones. There are two primary forms of Scan:

- **Inclusive Scan:** The partial reduction at each position *includes* the value of the current element.
 - Input: [3, 1, 4, 1] → Output: [3, 4, 8, 9]
- **Exclusive Scan:** The partial reduction at each position *excludes* the current element, only considering the elements before it. It is often initialized with an identity value (like 0 for addition).
 - Input: [3, 1, 4, 1] → Output: [0, 3, 4, 8]

Parallelizing the Scan

A clever two-phase algorithm is used to parallelize the scan operation, avoiding the serial dependency.

1. **Up Sweep (Reduction Phase):** In the first phase, a **reduction tree** is built from the input data, similar to a parallel reduce. This pass computes partial summaries over increasingly large blocks of the data but does not yet produce the final scan output.
2. **Down Sweep (Intermediate Value Phase):** In the second phase, the algorithm traverses back down the tree. Using the intermediate values computed during the up sweep, it calculates the correct prefix sum for each element in the output sequence. This phase effectively distributes the partial sums back across the collection.



(Image illustrating the two-phase up-sweep and down-sweep data flow of a parallel scan)

The time order is

$$O(2\log(n))$$

Optimization and Fusion

Just like Reduce, the Scan pattern can be optimized for large datasets using **tiling**. This involves a three-phase scan where local scans are performed on tiles, the results are adjusted based on a global scan of the tile summaries, and then the local results are updated.

Furthermore, Scan can also be **fused with a Map** pattern. A function is first applied to every element of the input, and then a scan is performed on the resulting collection.

Example Uses

The Scan pattern is a surprisingly powerful and versatile building block used in a wide variety of algorithms, including:

- **Sorting algorithms** like radix sort.

- **Lexical analysis** for parsing code into tokens.
- **Evaluating polynomials.**
- **Searching for regular expressions.**
- **Image processing** for labeling connected components.
- **Tree algorithms**, such as calculating the depth of every node in a tree.

F. Parallel Computing: Data Movement and Organization

In the world of parallel computing, performance is often limited more by the cost of moving data than by the speed of computation itself. Transferring data between memory layers (e.g., from RAM to CPU caches) or across networks is expensive in terms of both time and power. Therefore, for data-intensive applications like searching and sorting, it is crucial to design algorithms around efficient data movement patterns. This document explores fundamental patterns for parallel data reorganization: **Gather**, **Scatter**, and **Pack**, along with key concepts in data partitioning and layout.

#BOOKMARK

The Gather Pattern

Core Concept

The **Gather** pattern is a fundamental operation for parallel data reading. It constructs a new output collection by reading elements from a source collection at locations specified by an *index array*.

The process is straightforward:

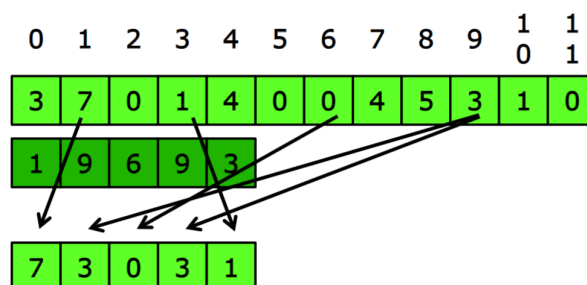
1. You have a source data collection (e.g., an array **S**).
2. You have a collection of indices (e.g., an array **I**).
3. The output collection **O** is created such that $O[i] = S[I[i]]$.

Essentially, Gather performs a number of random reads in parallel. Because reading from the same location multiple times is not a conflicting operation.

Gather is an **embarrassingly parallel** pattern.

Each read can be executed independently without the need for synchronization, making it highly efficient. The main performance bottleneck is the random memory access pattern, which can lead to *cache misses*.

The output collection will have the same number of elements and dimensionality as the index collection, and its elements will be the same type as the source collection.



(Image illustrating how an index array $[1, 5, 0, 2, 2]$ is used to "gather" elements from a source array $['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']$ into a new array $['B', 'F', 'A', 'C', 'C', 'E']$)

Special Cases of Gather

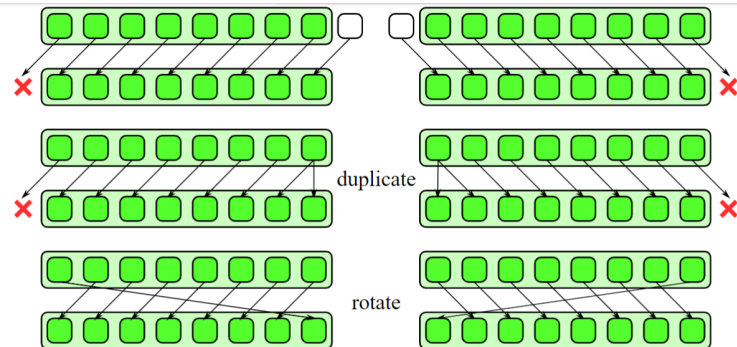
While the general Gather pattern involves random reads, several special cases have regular, predictable access patterns that can be heavily optimized.

1. Shifts

A **Shift** moves data elements to the left or right in memory by a fixed distance. Because the data access is regular and predictable (e.g., every thread reads from $\text{index} + C$), it benefits greatly from **data locality** and can be implemented very efficiently using **vector instructions** (SIMD), which process multiple data elements at once.

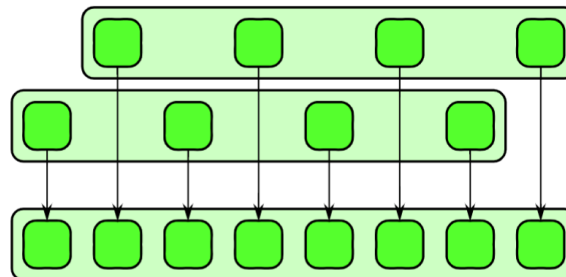
Variants of the shift pattern are defined by how they handle boundary conditions (the "empty" spots created at the edge of the array):

- **Rotate**: The elements shifted off one end wrap around to the other end.
- **Duplicate**: The boundary elements are duplicated to fill the empty spots.
- **Default Value**: The empty spots are filled with a specified default value.



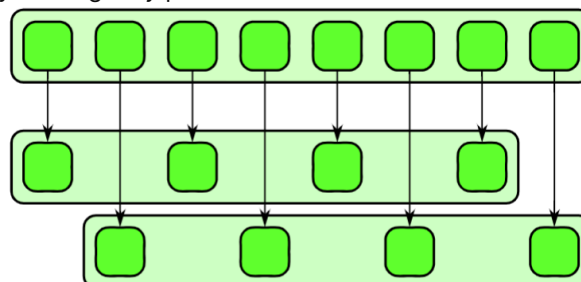
2. Zip

The **Zip** pattern interleaves elements from two or more input arrays into a single output array, like the teeth of a zipper. For example, it can take an array of real numbers and an array of imaginary numbers and combine them into a single array of complex number pairs. This can be generalized to interleave elements from multiple arrays and can handle arrays of different data types.



3. Unzip

Unzip is the inverse operation of Zip. It extracts elements from a single input array into two or more output arrays based on a regular offset and stride. For instance, it can take an array of complex numbers and separate it back into an array of real parts and an array of imaginary parts.



The Scatter Pattern

Core Concept

Scatter is the dual of Gather; it is a pattern for parallel *writing*. It takes an input collection and writes each of its elements to a destination collection at locations specified by an index map.

While the concept is simple, Scatter introduces a major challenge not present in Gather: **write collisions**, also known as **race conditions**. A collision occurs when two or more parallel threads attempt to write to the same memory location at the same time. Without rules to manage this, the final value at that location becomes undefined and depends on the unpredictable timing of the threads.

To make Scatter a deterministic and useful pattern, we must define **collision resolution rules**.

Collision Resolution Rules

1. **Atomic Scatter**: This is a non-deterministic approach. When a collision occurs, the hardware ensures that one write completes in its entirety without being corrupted by the others. However, there is **no guarantee as to which write will "win"**. The result is valid but unpredictable.
2. **Permutation Scatter**: This rule simply defines collisions as **illegal**. The index map must be a permutation, meaning every location is written to exactly once. This is useful for operations that reorder data without loss, such as matrix transposition or the scrambling stages of an FFT. If a collision-free pattern is known, it is often more efficient to convert the scatter into a gather.
3. **Merge Scatter**: This rule resolves collisions by combining the values being written using a specified **associative and commutative operator**. For example, if multiple threads attempt to write numbers to the same location, a merge scatter with an addition operator would add all the numbers together and write the sum. Both properties are required because the order of arrival of the writes is not guaranteed. This is the basis for building histograms in parallel.
4. **Priority Scatter**: In this scheme, each element in the input array is assigned a priority (often based on its position). When a collision occurs, the element with the **highest priority is written**, and the others are discarded. A key example is in 3D graphics rendering, where a Z-buffer uses a priority scatter to ensure that only the pixel closest to the camera is drawn at each screen coordinate.

Performance Considerations

Scatter is **generally a more expensive operation than Gather**. Random writes can cause significant cache coherency traffic, especially a problem known as **false sharing**, where different cores repeatedly invalidate the same cache line because they are writing to nearby, but separate, memory locations.

The Pack Pattern

Core Concept

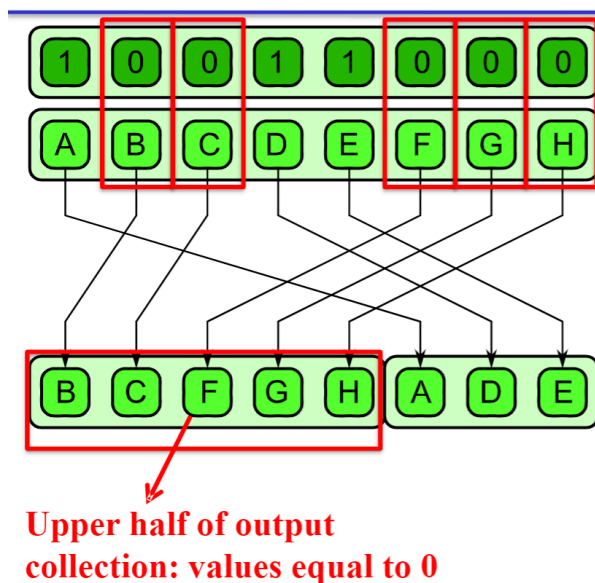
The **Pack** operation is used to **filter a collection**.

Given an input array and a corresponding flag array (e.g., booleans or 0s and 1s), Pack creates a new, smaller array containing only those elements from the input marked for retention.

The key benefit is that the **retained elements become contiguous in memory, which greatly improves the performance of subsequent computations by improving cache utilization and enabling vectorization**.

A highly efficient parallel algorithm for Pack uses a **Scan** operation:

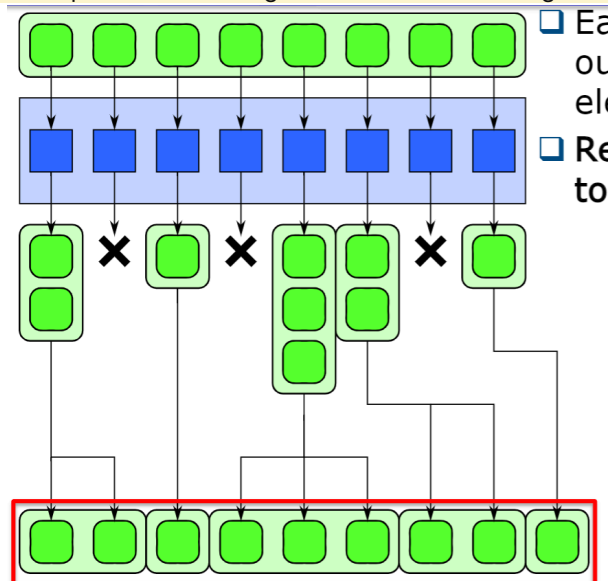
1. Convert the boolean flag array into an array of integers (0s and 1s).
2. Perform an **exclusive scan** with addition on this integer array. The result gives the new destination index for each element that will be kept.
3. Use a scatter-like operation to write the retained values to their new, compacted locations.



(Image showing how a flag array is used to "pack" an array, removing elements marked with 0 and compacting the rest.)

Generalizations of Pack

1. **Split:** This is a generalization that does not discard data. Instead, it partitions the elements into two groups based on a flag (e.g., true/false) and moves them to different halves of the output array.
2. **Bin (or Bucket):** This extends Split to support more than two categories. Elements are partitioned into multiple "bins" based on a key or category. This is a core component of algorithms like **Radix Sort**.
3. **Expand:** This is the most general form, where each input element can produce zero, one, or any number of output elements. The final results from all inputs are fused together in order into a single contiguous output array.



Data Partitioning and Layout

Partitioning Strategies

A common strategy for parallelizing algorithms is to divide the data into sections, work on the sections independently, and then combine the results. Two ways to divide data are:

- **Partitioning:** Dividing data into **equal-sized**, non-overlapping regions.
- **Segmentation:** Dividing data into **non-uniform**, non-overlapping regions.

Array of Structures (AoS) vs. Structure of Arrays (SoA)

The way data is laid out in memory is critical for performance. For complex data objects (structs), there are two primary layouts:

1. **Array of Structures (AoS):** This is the most intuitive layout, where an array holds entire **struct** objects.
 - `struct { float x, y, z; } points[N];`
 - **Pros:** Good for random access where you need all fields of a **single** object at once.
 - **Cons:** Terrible for parallel operations that work on one field at a time (e.g., calculating the average of all **x** coordinates). Data for a single field is spread out in memory, which prevents efficient **vectorization** and leads to poor cache usage.
2. **Structure of Arrays (SoA):** This layout inverts the structure. A single **struct** contains arrays, with each array holding one field for all objects.
 - `struct { float x[N], y[N], z[N]; } points;`
 - **Pros:** Typically much better for parallel computing. All data for a single field (e.g., all **x** coordinates) is contiguous in memory. This is ideal for vector (SIMD) instructions and avoids cache problems like false sharing.
 - **Cons:** Accessing all the fields for a single object requires fetching data from multiple different arrays.

For most high-performance parallel tasks, **SoA is the preferred data layout** because it aligns with how modern hardware processes data in parallel chunks.

G. A Comprehensive Overview of Heterogeneous Computing, DSLs, and High-Level Synthesis

The Imperative for Heterogeneous Computing

Modern computing applications, from cloud services to embedded devices, present complex and **varied workloads**. These workloads are rarely uniform; they consist of parts that are easily parallelizable and parts that are difficult to parallelize, they exhibit both predictable and unpredictable data access patterns, and they combine simple data-parallel (SIMD) operations with complex, divergent control flows.

Attempting to execute these diverse tasks on a single type of general-purpose processor is inherently inefficient.

o Oon olO

The optimal solution is a **heterogeneous system**, an architecture composed of different types of processing cores, each specialized for a particular kind of task. Such a system might include:

- **Latency Cores:** Optimized for fast single-thread performance.
- **Throughput Cores:** Designed for massive parallel data processing.
- **DSP Cores:** Specialized for digital signal processing tasks.
- **Hardware IPs (Intellectual Property):** Fixed-function blocks for specific tasks like video encoding/decoding.
- **Configurable Logic:** Programmable hardware like FPGAs.
- **On-chip Memories:** Various levels of cache and local storage.

The Challenge of Energy Efficiency

A primary driver for heterogeneous computing is the critical need for **energy efficiency**. Given a fixed power budget, the goal is not just to maximize performance, but to maximize performance **per watt**. **General-purpose** processors are notoriously inefficient in this regard. *For any given instruction, a significant portion of the energy is consumed not by the actual arithmetic operation, but by the overhead required to support its generality:*

- **Instruction Supply (42%):** Fetching the instruction from memory, address translation, and managing the instruction cache.
- **Data Supply (28%):** Accessing operands from the register file or memory.
- **Clock and Control (24%):** Managing pipeline hazards, dependencies, and decoding instructions into micro-operations.
- **Arithmetic (6%):** The actual computation.

This overhead demonstrates that specialization is key. A simple hardware circuit designed for a single task, like a logical OR operation, is vastly more energy-efficient than a general-purpose processor executing the same operation.

Furthermore, data movement is extremely costly in terms of energy. An arithmetic operation (ALU op) might cost a single picojoule (pJ), whereas moving data off-chip can cost 500 times that, sending it to DRAM costs 1,000 times more, and sending it over a wireless network like LTE can cost over 10,000,000 times more. Therefore, two core principles for energy-efficient computing are:

1. **Use specialized processors.**
2. **Move as little data as possible.**

The Spectrum of Processing Hardware

Choosing the right tool for the job involves a trade-off between flexibility and efficiency:

- **CPUs (Control-oriented):** Offer maximum flexibility and are the easiest to program. They are ideal for complex control flow and serial tasks but are the least efficient in terms of performance per watt.
- **GPUs (Throughput-oriented):** Offer a significant step up in efficiency (around 10x) for highly parallel, data-intensive tasks. They are more difficult to program than CPUs.
- **FPGAs (Reconfigurable):** Field-Programmable Gate Arrays provide a large increase in efficiency (potentially 100x) by allowing logic to be configured in hardware. They are notoriously difficult to program, though active research aims to simplify this.
- **ASICs (Fixed function):** Application-Specific Integrated Circuits represent the pinnacle of efficiency (100-1000x more efficient than CPUs). They are custom-designed for a single task, are not programmable, and have extremely high non-recurring engineering costs for design and manufacturing.

Programming Heterogeneous Systems

The central challenge is how to write **efficient, portable, and productive** parallel programs for these complex, emerging architectures. The ideal parallel programming language should provide a perfect balance of three key attributes:

- **Performance:** The ability to extract the maximum capability from the hardware.
- **Productivity:** The ease and speed with which a developer can write, debug, and maintain correct code.
- **Generality:** The ability of the language to express a wide range of algorithms and target different hardware architectures.

In practice, these three goals are in conflict. Languages like C++ and Java offer generality but achieving high performance requires significant manual effort. Languages like Python and Ruby offer high productivity but often sacrifice performance.

Domain-Specific Languages (DSLs)

A powerful solution to this challenge is the use of **Domain-Specific Languages (DSLs)**. A DSL is a programming language with restricted expressiveness, tailored specifically for a particular problem domain. Key characteristics of DSLs include:

- **High-level and Declarative:** Developers describe *what* they want to compute, not *how* to compute it.
- **Deterministic:** The same code will always produce the same output.
- **Embedded or External:** They can be standalone languages or, more commonly, libraries embedded within a general-purpose host language like C++ or Python.

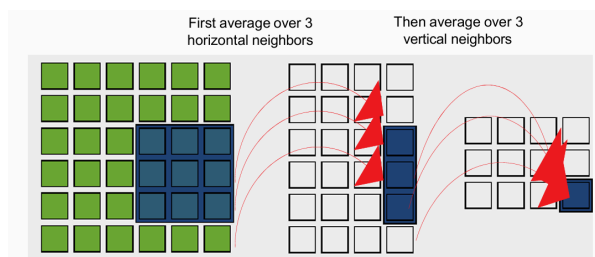
By focusing on a narrow domain, a DSL can offer both high performance and high productivity, sacrificing only generality. Examples include SQL for databases, MATLAB for numerical computing, PyTorch for machine learning, and OpenGL for graphics.

Case Study: Image Processing with Halide

Image processing is a domain that perfectly illustrates the need for extreme optimization of performance and power consumption. Writing efficient image processing pipelines is hard because it requires manually transforming the program's structure to exploit **parallelism** (distributing work) and **locality** (keeping data in cache).

The Complexity of Manual Optimization

Consider a simple C++ implementation of a 3×3 box blur. A naive implementation first computes a horizontal blur across the entire image and stores the intermediate result in memory (**blurx**), and then loads that result back to compute a vertical blur. On a quad-core CPU, this might take around **9.96 ms/megapixel**.



```

void box_filter_3x3(const Image &in, Image &blury){
    Image blurx(in.width(), in.height()); // allocate blurx array

    for (int y = 0; y < in.height(); y++)
        for (int x = 0; x < in.width(); x++)
            blurx(x, y) = (in(x-1, y) + in(x, y) + in(x+1, y))/3;

    for (int y = 0; y < in.height(); y++)
        for (int x = 0; x < in.width(); x++)
            blury(x, y) = (blurx(x, y-1) + blurx(x, y) + blurx(x, y+1))/3;
}

```

To optimize **this**, a developer must manually restructure the code to improve performance. This involves:

1. ****Parallelism:**** Using compiler **pragmas** (like OpenMP) to distribute work among threads and using **SIMD** (Single Instruction, Multiple Data) vector intrinsics to process multiple pixels at once.
2. ****Locality:**** Instead of processing the whole image at once, the computation is broken into small tiles. The horizontal blur **for** a tile is computed, and immediately after, the vertical blur is computed on that same tile. This keeps the intermediate data in the local cache, avoiding slow main memory access.

```

` `` ` cpp
void box_filter_3x3(const Image &in, Image &blury) {
    __m128i one_third = _mm_set1_epi16(21846);
    #pragma omp parallel for
    for (int yTile = 0; yTile < in.height(); yTile += 32) {
        __m128i a, b, c, sum, avg;
        __m128i blurx[(256/8)*(32+2)]; // allocate tile blurx array
        for (int xTile = 0; xTile < in.width(); xTile += 256) {
            __m128i *blurxPtr = blurx;
            for (int y = -1; y < 32+1; y++) {
                const uint16_t *inPtr = &(in[yTile+y][xTile]);
                for (int x = 0; x < 256; x += 8) {
                    a = _mm_loadu_si128((__m128i*)(inPtr-1));
                    b = _mm_loadu_si128((__m128i*)(inPtr+1));
                    c = _mm_load_si128((__m128i*)(inPtr));
                    sum = _mm_add_epi16(_mm_add_epi16(a, b), c);
                    avg = _mm_mulhi_epi16(sum, one_third);
                    _mm_store_si128(blurxPtr++, avg);
                    inPtr += 8;
                }
            }
            blurxPtr = blurx;
            for (int y = 0; y < 32; y++) {
                __m128i *outPtr = (__m128i*)&(blury[yTile+y][xTile]);
                for (int x = 0; x < 256; x += 8) {
                    a = _mm_load_si128(blurxPtr+(2*256)/8);
                    b = _mm_load_si128(blurxPtr+256/8);
                    c = _mm_load_si128(blurxPtr++);
                    sum = _mm_add_epi16(_mm_add_epi16(a, b), c);
                    avg = _mm_mulhi_epi16(sum, one_third);
                    _mm_store_si128(outPtr++, avg);
                }
            }
        }
    }
}

```

```

    }
  }
}

```

This manually optimized version is much faster—achieving around **0.9 ms/megapixel** (an 11x speedup). However, the code becomes incredibly complex, difficult to read, hard to debug, and is not portable to different hardware (e.g., a GPU). Exploring different optimization strategies (like changing the tile size) is a time-consuming and manual process.

Halide's Approach: Decoupling Algorithm from Schedule

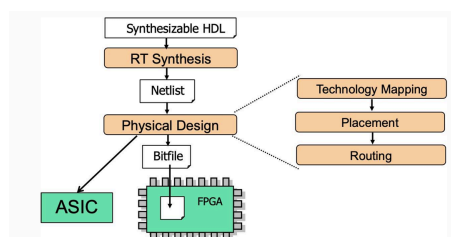
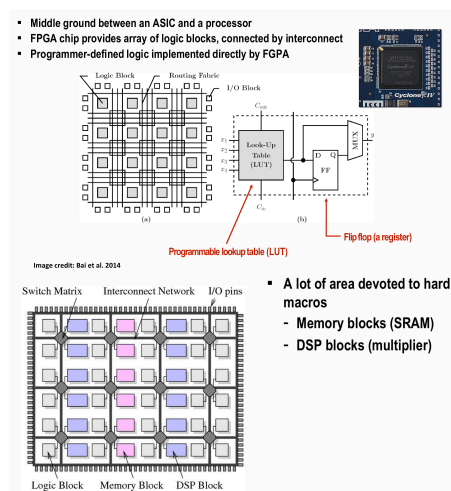
Halide is a DSL for image processing that solves this problem with a radical approach: it **decouples the algorithm from its schedule**.

- **The Algorithm:** This is the pure, mathematical definition of *what* is being computed. It's a simple, high-level description of the value of each pixel in the output image as a function of the input. This part of the code is written once and rarely changes.
- **The Schedule:** This defines *where* and *when* the computation happens. It is a separate part of the code that specifies the optimization strategy. The developer can command Halide to tile the computation, vectorize it, parallelize it across CPU cores, or even offload it to a GPU.

This separation is incredibly powerful. The developer can maintain a simple, clean, and readable algorithm definition while experimenting with **dozens of complex optimization schedules without ever touching the core logic**. Halide makes the iterative process of performance tuning significantly easier and allows the same algorithm to be efficiently compiled for entirely different hardware architectures simply by changing the schedule.

An Introduction to High-Level Synthesis (HLS)

While DSLs like Halide address the programmability of CPUs and GPUs, High-Level Synthesis (HLS) targets the **more efficient end of the hardware spectrum**: FPGAs and ASICs.



From Traditional RTL to High-Level Design

Traditionally, designing hardware for FPGAs or ASICs requires using a **Hardware Description Language (HDL)** like Verilog or VHDL to perform **Register-Transfer Level (RTL) design**. This involves manually specifying the behavior of individual components like multiplexers and registers and how they are connected. RTL design gives precise, fine-grained control but is extremely time-consuming, error-prone, and requires specialized hardware design expertise.

High-Level Synthesis is a transformative technology that automates this process. It functions like a compiler, but for hardware. HLS tools take a high-level behavioral description written in a standard programming language (like C, C++, or SystemC) and automatically translate it into an RTL implementation (Verilog/VHDL).

The benefits of HLS are significant:

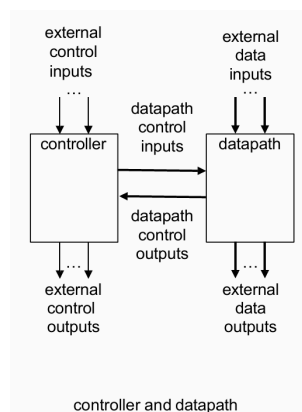
- **Productivity:** The pool of expert software developers is much larger than that of hardware designers.
- **Abstraction:** It's easier to describe complex functionalities in a high-level language.
- **Portability:** The design is technology-independent and can be re-targeted to different hardware with less effort.
- **Faster Design Cycle:** Automation dramatically speeds up both design and verification.

However, HLS also presents challenges. Unlike a software compiler that maps instructions to an existing, fixed architecture, an HLS tool must *create* a custom hardware architecture, which involves exploring a vast and complex design space.

The FSMD Model

The architecture generated by HLS tools is typically a **Finite State Machine with Datapath (FSMD)**. This model consists of two main parts:

- **Datapath:** This is the "brawn" (muscles) of the circuit. It contains the functional resources that perform operations (e.g., adders, multipliers), memory resources to store data (registers, RAMs), and interconnection resources (multiplexers, buses) to route data between them.
- **Controller:** This is the "brain." It is a Finite State Machine (FSM) that issues control signals to the datapath on each clock cycle, dictating which operations to perform, where to read data from, and where to write the results.

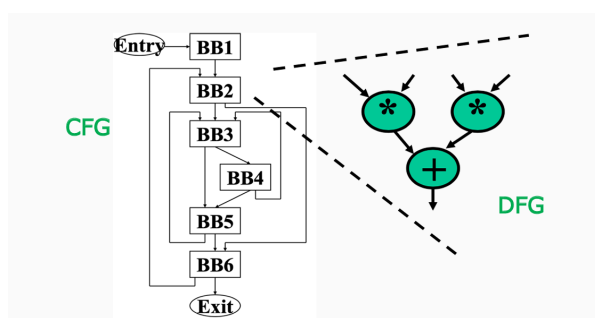


Deep Dive: HLS Scheduling

To get from high-level code to an FSMD, an HLS tool must perform several automated steps, the most crucial of which is **scheduling**.

Internal Representations for HLS

1. Before scheduling can occur, the HLS compiler first converts the input code into an **Intermediate Representation (IR)** that makes dependencies explicit.
2. A **Control Flow Graph (CFG)** represents the high-level structure (loops, conditionals), where nodes are basic blocks of instructions.
3. Within each basic block, a **Data Flow Graph (DFG)** is created, where nodes represent operations and edges represent the data dependencies between them.



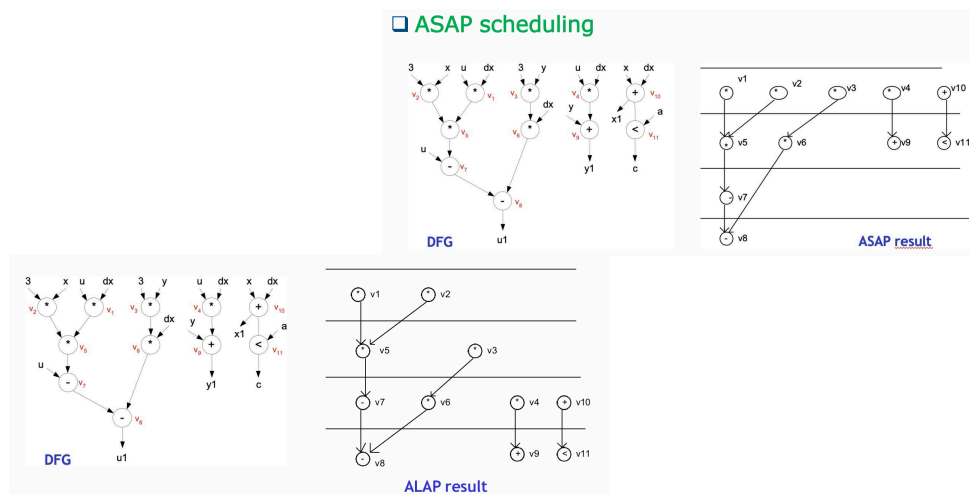
The Scheduling Problem

Scheduling is the process of assigning each operation from the DFG to a specific time slot, or **control step** (which usually corresponds to a clock cycle). The goal of the scheduler is to find an assignment that meets certain objectives, such as minimizing latency (the total number of clock cycles) for a given set of hardware resources, or minimizing the hardware resources required to meet a specific latency target.

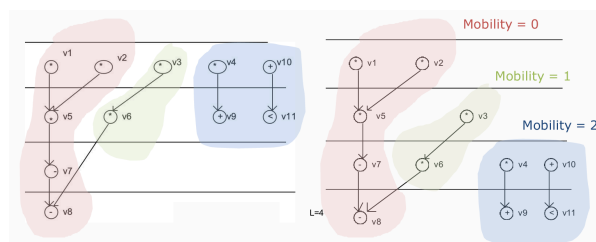
Scheduling is an **NP-hard problem**, meaning that finding the absolute optimal solution is computationally intractable for all but the simplest cases. Therefore, HLS tools rely on a variety of efficient heuristics.

Fundamental Scheduling Algorithms

- **ASAP and ALAP Scheduling:** The two most basic scheduling algorithms are As-Soon-As-Possible (ASAP) and As-Late-As-Possible (ALAP).
 - **ASAP scheduling** places each operation in the earliest possible control step, respecting data dependencies.
 - **ALAP scheduling**, given a total latency constraint (often the one found by ASAP), places each operation in the latest possible control step.



- **Mobility:** The difference between an operation's ALAP start time and its ASAP start time is its **mobility**. An operation with **zero mobility** is on the **critical path** of the computation; delaying it will delay the entire computation. Operations with higher mobility are more flexible and can be moved around to optimize resource usage.



List-Based Scheduling: A Practical Heuristic

List-based scheduling is a common and effective greedy heuristic used in HLS. The general algorithm is as follows:

1. **Construct a Priority List:** All operations are placed in a list and prioritized. A common and effective priority function is mobility—operations with lower mobility (more critical) are given higher priority.
2. **Iterate Through Control Steps:** The scheduler proceeds one control step at a time, starting from the first.
3. **Assign Operations:** In each control step, the scheduler examines a "ready list" of operations whose data dependencies have been met. From this list, it selects operations to schedule based on their priority and the availability of hardware resources (e.g., if there is only one multiplier, only one multiplication can be scheduled).
4. **Update and Repeat:** Once operations are assigned to the current control step, they are removed from the priority list, and the ready list is updated for the next cycle. This process continues until all operations have been scheduled.

This method does not guarantee an optimal solution, but it is computationally efficient and produces high-quality results in practice, enabling the automated translation from a high-level algorithm into a detailed, cycle-by-cycle hardware implementation.

Of course. Here is a detailed transcription and step-by-step explanation of the HLS list-based scheduling exercise presented in the PDF.

Extended example

This exercise demonstrates how a High-Level Synthesis (HLS) tool schedules a given block of code onto hardware using the **list-based scheduling** algorithm. The goal is to assign each operation to a specific clock cycle while respecting data dependencies and resource constraints.

1. Problem Definition

We are given a basic block of code and a set of hardware constraints.

Input Code (HLS IR):

```
tmp_1 = a * b;
tmp_2 = c * d;
x_1 = tmp_1 * tmp_2;
x_2 = a * d;
x_3 = x_2 - x_1;
x_4 = x_2 + x_3;
x_5 = c * 3;
x_6 = x_4 + x_4;
x_7 = tmp_1 * x_5;
out_1 = x_6 * x_7;
```

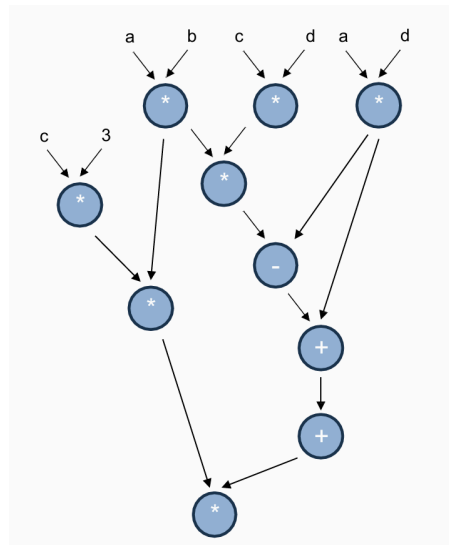
C++

Hardware Constraints:

- **Available Functional Units (FUs):**
 - 1 **Adder** with a latency of **1 clock cycle**.
 - 1 **Multiplier** with a latency of **2 clock cycles**.
 - 1 **ALU** (Arithmetic Logic Unit) that can perform any arithmetic operation, with a latency of **2 clock cycles**.
 - **Priority Method:** The priority of an operation is determined by its **mobility**. Operations with lower mobility have higher priority.
 - **Inputs:** **a**, **b**, **c**, and **d** are primary inputs, available at the beginning (cycle 0).
-

2. Building the Data Flow Graph (DFG)

The first step is to convert the linear code into a Data Flow Graph. The DFG represents operations as nodes and the data dependencies between them as directed edges. An edge from operation A to operation B means that B cannot start until A has finished.



This graph makes the potential parallelism explicit. For example, `tmp_1`, `tmp_2`, `x_2`, and `x_5` can all potentially start at the same time because their inputs are all available from the beginning.

3. Calculating Mobility

Mobility determines the scheduling flexibility of an operation. It is calculated as the difference between an operation's latest possible start time and its earliest possible start time.

ol

To find this, we perform two initial scheduling passes without resource constraints:

1. **ASAP (As Soon As Possible) Scheduling:** Each operation is scheduled at the earliest possible clock cycle, determined by when its input data becomes available.
2. **ALAP (As Late As Possible) Scheduling:** Given the total latency found by ASAP, each operation is scheduled at the latest possible clock cycle without increasing the total latency.

By calculating the ASAP and ALAP start times for each operation, we derive the following mobility values. This list now serves as our **priority list**, where **0 is the highest priority**.

Operation	Mobility	Priority
<code>tmp_1</code>	0	Highest
<code>tmp_2</code>	0	Highest
<code>x_1</code>	0	Highest
<code>x_3</code>	0	Highest
<code>x_4</code>	0	Highest
<code>x_6</code>	0	Highest
<code>out_1</code>	0	Highest
<code>x_2</code>	2	Medium
<code>x_5</code>	3	Low
<code>x_7</code>	3	Low

4. The List-Based Scheduling Process

Now we proceed cycle-by-cycle, scheduling operations from a "ready list" based on priority and available hardware.

Clock Cycle 1:

- **Ready List:** `tmp_1`, `tmp_2`, `x_2`, `x_5` (all their inputs are available).
- **Priorities:** `tmp_1` (0), `tmp_2` (0), `x_2` (2), `x_5` (3).
- **Scheduling Decision:**
 - We have two high-priority (mobility 0) multiplication operations: `tmp_1` and `tmp_2`.
 - We have two available multiplication units: the Multiplier and the ALU.
 - Assign `tmp_1 = a * b` to the **Multiplier**.
 - Assign `tmp_2 = c * d` to the **ALU**.
- **Note:** Both operations have a 2-cycle latency, so these FUs will be busy for cycles 1 and 2.

Clock cycle	ADDER	MULTIPLIER	ALU
1		<code>tmp_1 = a * b;</code>	<code>tmp_2 = c * d;</code>
2		busy	busy

Clock Cycle 2:

- **Ready List:** `x_2`, `x_5`.
- **Scheduling Decision:** The Multiplier and ALU are **busy** executing the operations from cycle 1. No new operations can be scheduled.

Clock Cycle 3:

- **Ready List:** The results of `tmp_1` and `tmp_2` are now available. The ready list is: `x_1` (inputs `tmp_1`, `tmp_2` are ready), `x_2`, `x_5`.
- **Priorities:** `x_1` (0), `x_2` (2), `x_5` (3).
- **Scheduling Decision:**
 - `x_1` has the highest priority. It is a multiplication.
 - `x_2` is the next highest priority multiplication.
 - Assign `x_1 = tmp_1 * tmp_2` to the **Multiplier**.
 - Assign `x_2 = a * d` to the **ALU**.
- **Note:** The FUs will be busy for cycles 3 and 4.

Clock cycle	ADDER	MULTIPLIER	ALU
1		<code>tmp_1 = a * b;</code>	<code>tmp_2 = c * d;</code>
2		busy	busy
3		<code>x_1 = tmp_1 * tmp_2;</code>	<code>x_2 = a * d;</code>
4		busy	busy

Clock Cycle 4:

- **Ready List:** `x_5`.
- **Scheduling Decision:** The Multiplier and ALU are **busy**.

Clock Cycle 5:

- **Ready List:** Results of `x_1` and `x_2` are available. The ready list is: `x_3` (inputs `x_1`, `x_2` ready), `x_5`.
- **Priorities:** `x_3` (0), `x_5` (3).
- **Scheduling Decision:**
 - `x_3` has the highest priority. It is a subtraction. Assign `x_3 = x_2 - x_1` to the **Adder** (1-cycle latency).
 - `x_5` is also ready. It is a multiplication. Assign `x_5 = c * 3` to the **Multiplier** (2-cycle latency).

Clock Cycle 6:

- **Ready List:** The result of `x_3` is now available. The ready list is: `x_4` (input `x_3` ready).
- **Priorities:** `x_4` (0).
- **Scheduling Decision:**
 - Assign `x_4 = x_2 + x_3` to the **Adder**.
 - The Multiplier is **busy** with `x_5`.

Clock cycle	ADDER	MULTIPLIER	ALU
1		tmp_1 = a * b;	tmp_2 = c * d;
2		busy	busy
3		x_1 = tmp_1 * tmp_2;	x_2 = a * d;
4		busy	busy
5	x_3 = x_2 - x_1;	x_5 = c * 3;	
6	x_4 = x_2 + x_3;	busy	

Clock Cycle 7:

- **Ready List:** Results of **x_4** and **x_5** are available. Ready list: **x_6**, **x_7**.
- **Priorities:** **x_6** (0), **x_7** (3).
- **Scheduling Decision:**
 - Assign **x_6 = x_4 + x_4** to the **Adder**.
 - Assign **x_7 = tmp_1 * x_5** to the **Multiplier** (the ALU is also free but we use the dedicated multiplier).

Clock cycle	ADDER	MULTIPLIER	ALU
1		tmp_1 = a * b;	tmp_2 = c * d;
2		busy	busy
3		x_1 = tmp_1 * tmp_2;	x_2 = a * d;
4		busy	busy
5	x_3 = x_2 - x_1;	x_5 = c * 3;	
6	x_4 = x_2 + x_3;	busy	
7	x_6 = x_4 + x_4;	x_7 = tmp_1 * x_5;	
8		busy	

Clock Cycle 8:

- **Ready List:** Empty.
- **Scheduling Decision:** The Adder is free. The Multiplier is **busy** with **x_7**.

Clock Cycle 9:

- **Ready List:** Results of **x_6** and **x_7** are available. The ready list is: **out_1**.
- **Priorities:** **out_1** (0).
- **Scheduling Decision:**
 - Assign **out_1 = x_6 * x_7** to the **Multiplier** (or ALU).

Clock Cycle 10:

- **Ready List:** Empty.
- **Scheduling Decision:** The Multiplier is **busy** with **out_1**.

The process is now complete.

5. Final Result: The Schedule

The final schedule assigns every operation to a start cycle, respecting all dependencies and resource constraints. The total latency is **10 clock cycles**.

Clock Cycle	ADDER	MULTIPLIER	ALU
1		tmp_1 = a * b	tmp_2 = c * d
2		busy	busy
3		x_1 = tmp_1 * tmp_2	x_2 = a * d
4		busy	busy
5	x_3 = x_2 - x_1	x_5 = c * 3	
6	x_4 = x_2 + x_3	busy	
7	x_6 = x_4 + x_4	x_7 = tmp_1 * x_5	
8		busy	
9		out_1 = x_6 * x_7	

Clock Cycle	ADDER	MULTIPLIER	ALU
10		busy	